

Deep Learning for Text Analytics

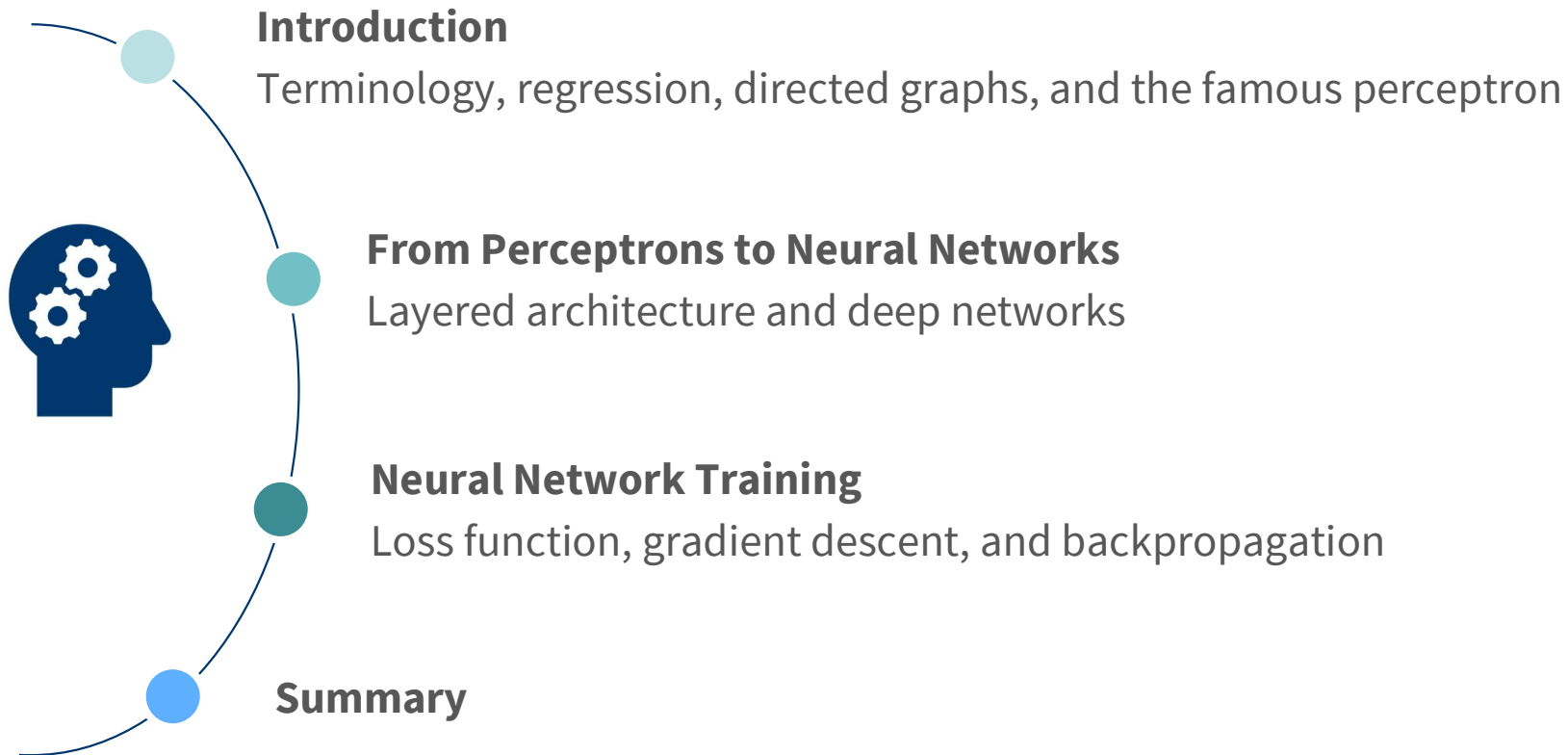


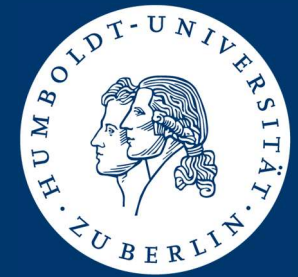
Foundations of Deep Learning (Part 1)

Stefan Lessmann [stefan.lessmann@hu-berlin.de]



Agenda





Introduction

Terminology, regression, directed graphs, and the famous perceptron

Machine Learning (ML) and Artificial Intelligence (AI)

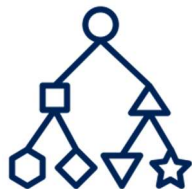
Artificial Intelligence

Enable computers to mimic human behavior



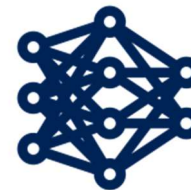
Machine Learning

Ability to learn without explicitly being programmed



Deep Learning

Ability to automatically extract features from data using artificial neural networks

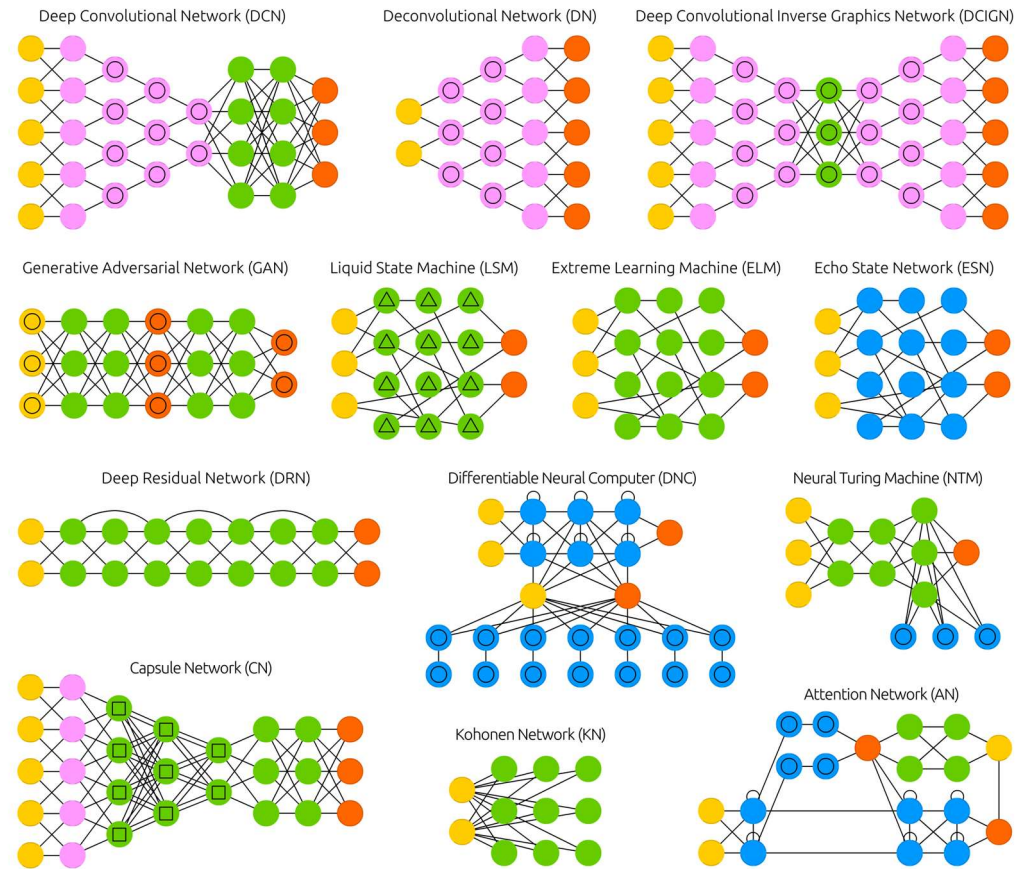
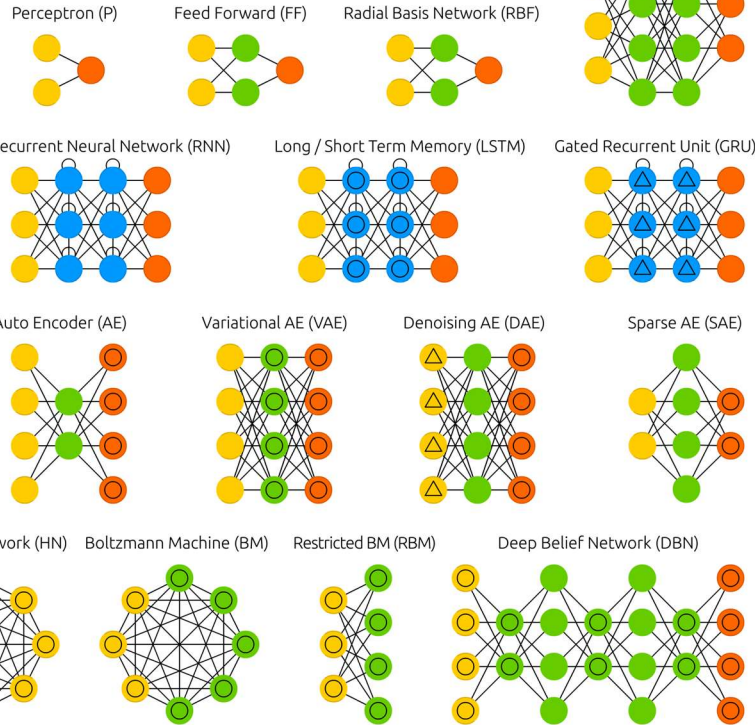


Deep Learning is as a Family of Neural Network-based Methods

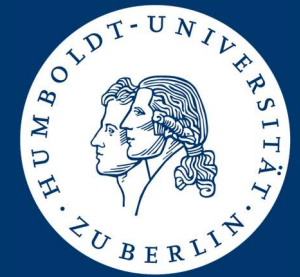
By F. Van Veen and S. Leijnen from The Asimov Institute

A mostly complete chart of Neural Networks

©2019 Fjodor van Veen & Stefan Leijnen asimovinstitute.org



Source: Van Veen, F. & Leijnen, S. (2019). The Neural Network Zoo. Retrieved from <https://www.asimovinstitute.org/neural-network-zoo>



Perceptrons and Neural Networks

Layered architecture, nonlinearity and deep networks

Two stage paradigm of Supervised ML

Training



Data-driven development of the mapping function

i	Y	X_1	X_2	...	X_m
1	...	...	...	...	...
2	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
n	...	...	...	...	...



Learning
Algorithm

Model

Testing

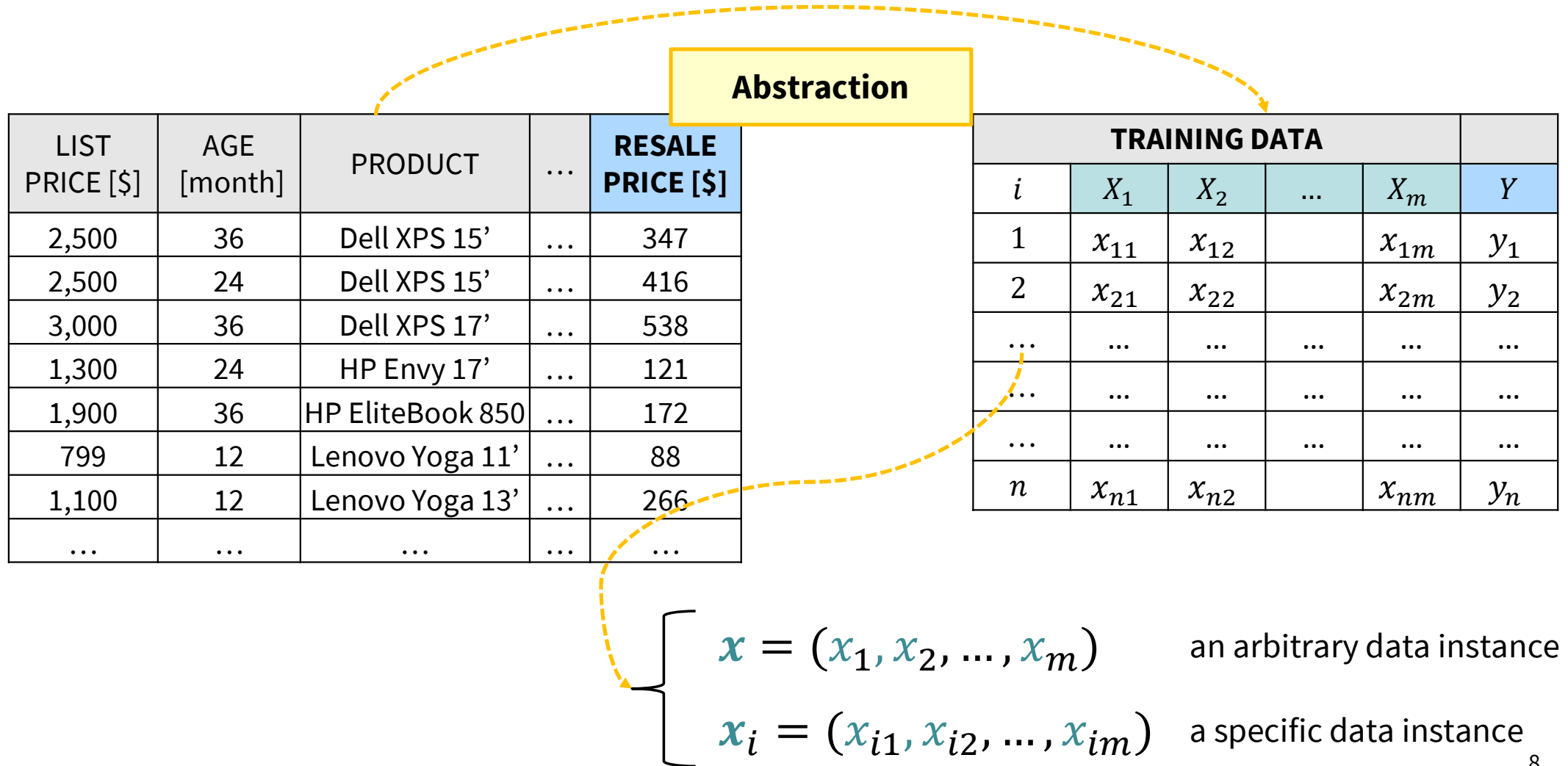


Apply learnt mapping function (i.e., model) to new observations

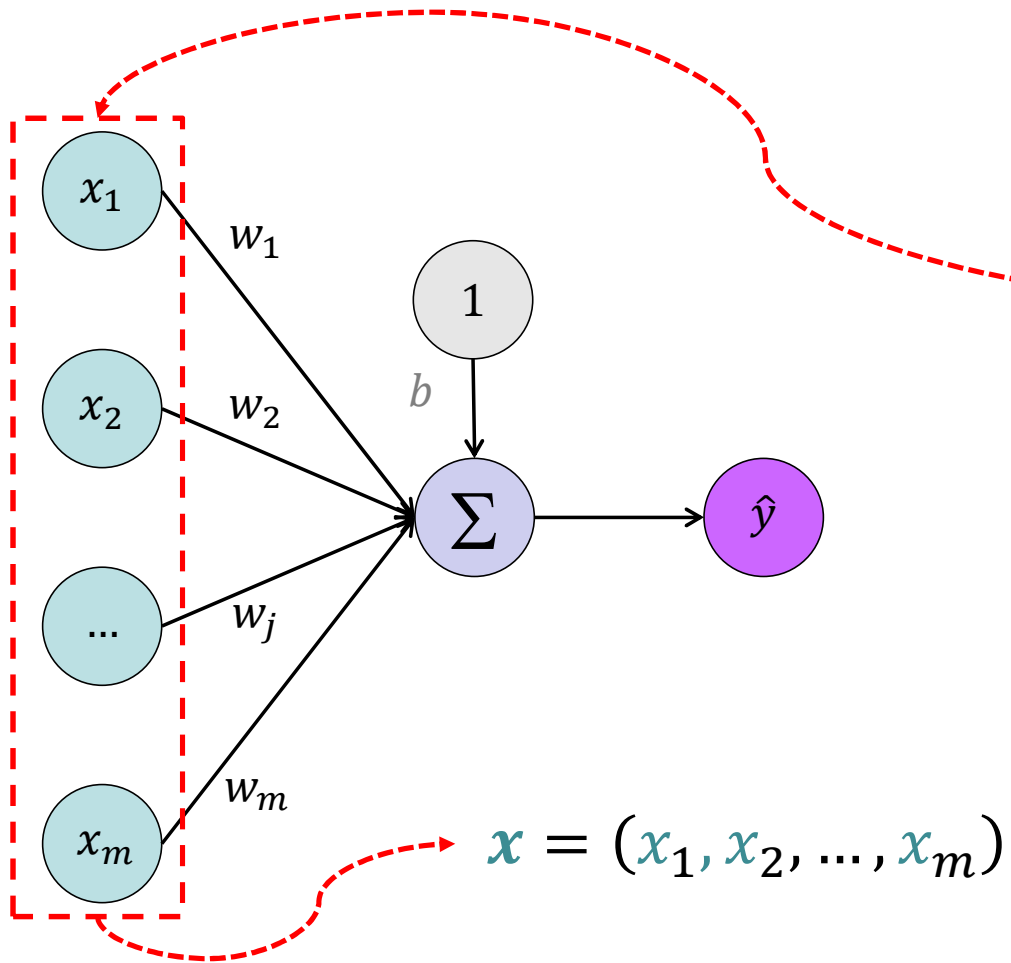
i	X_1	X_2	...	X_m
$n + 1$	...	...	...	...
$n + 2$	...	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
N	...	...	...	...

i	$\hat{Y}$
$n + 1$	...
$n + 2$	...
...	...
...	...
...	...
N	...

Data for Training Supervised ML Models for Regression

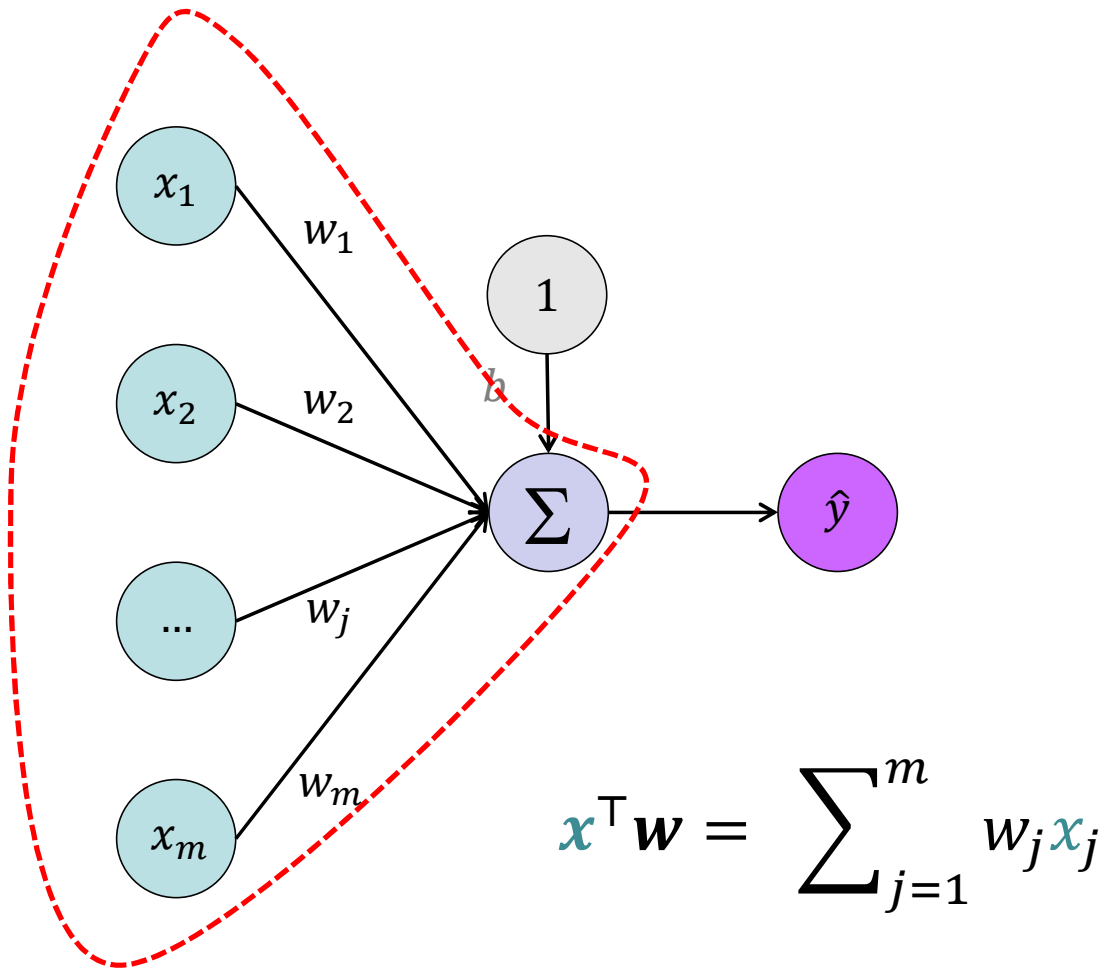


Linear Regression as Directed Graph



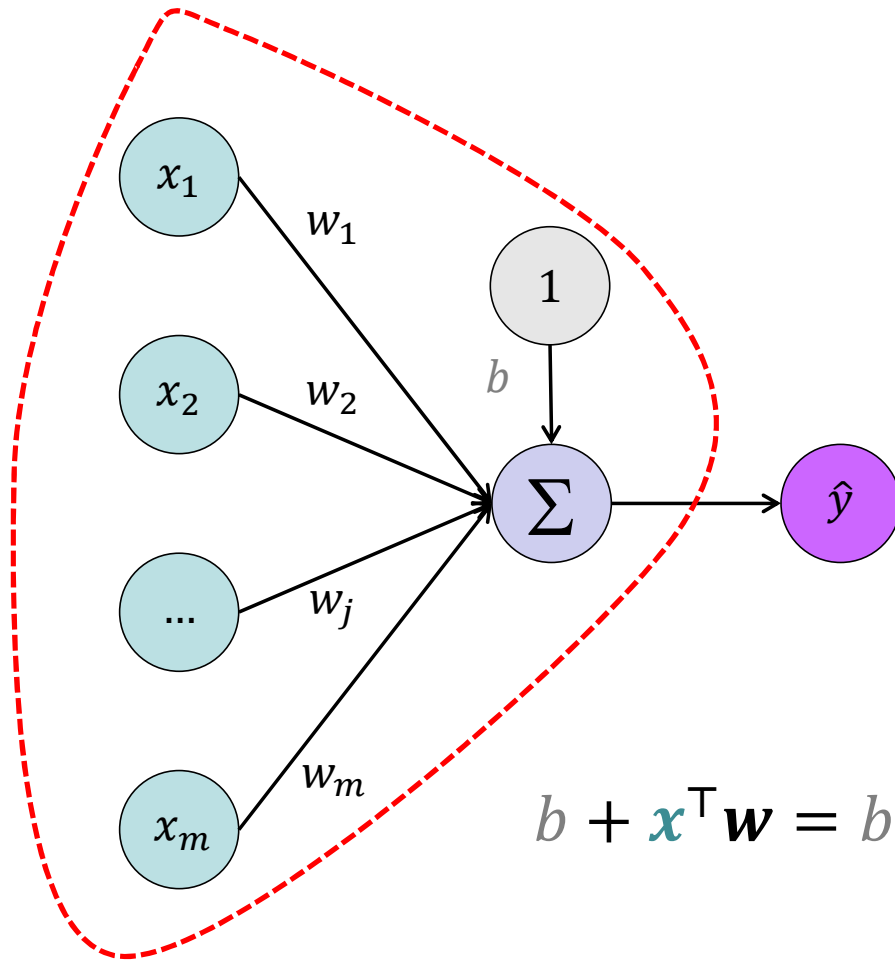
TRAINING DATA						
i	X_1	X_2	$\dots$	X_m	Y	$\hat{Y}$
1	x_{11}	x_{12}	$\dots$	x_{1m}	y_1	$\hat{y}_1$
2	x_{21}	x_{22}	$\dots$	x_{2m}	y_2	$\hat{y}_2$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
n	x_{n1}	x_{n2}	$\dots$	x_{nm}	y_n	$\hat{y}_n$

Linear Regression as Directed Graph



TRAINING DATA						
i	X_1	X_2	$\dots$	X_m	Y	$\hat{Y}$
1	x_{11}	x_{12}	$\dots$	x_{1m}	y_1	$\hat{y}_1$
2	x_{21}	x_{22}	$\dots$	x_{2m}	y_2	$\hat{y}_2$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
n	x_{n1}	x_{n2}	$\dots$	x_{nm}	y_n	$\hat{y}_n$

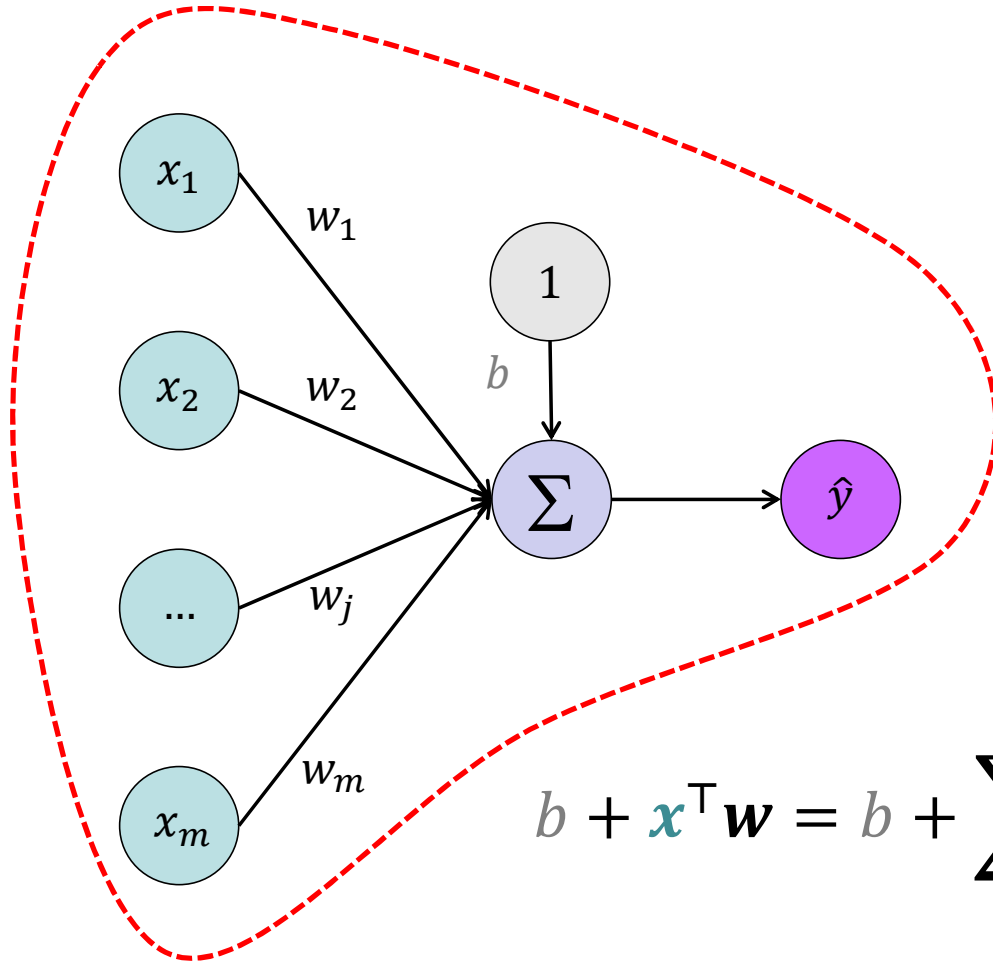
Linear Regression as Directed Graph



$$b + \mathbf{x}^\top \mathbf{w} = b + \sum_{j=1}^m w_j x_j$$

TRAINING DATA						
i	X_1	X_2	...	X_m	Y	$\hat{Y}$
1	x_{11}	x_{12}	...	x_{1m}	y_1	$\hat{y}_1$
2	x_{21}	x_{22}	...	x_{2m}	y_2	$\hat{y}_2$
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...
n	x_{n1}	x_{n2}	...	x_{nm}	y_n	$\hat{y}_n$

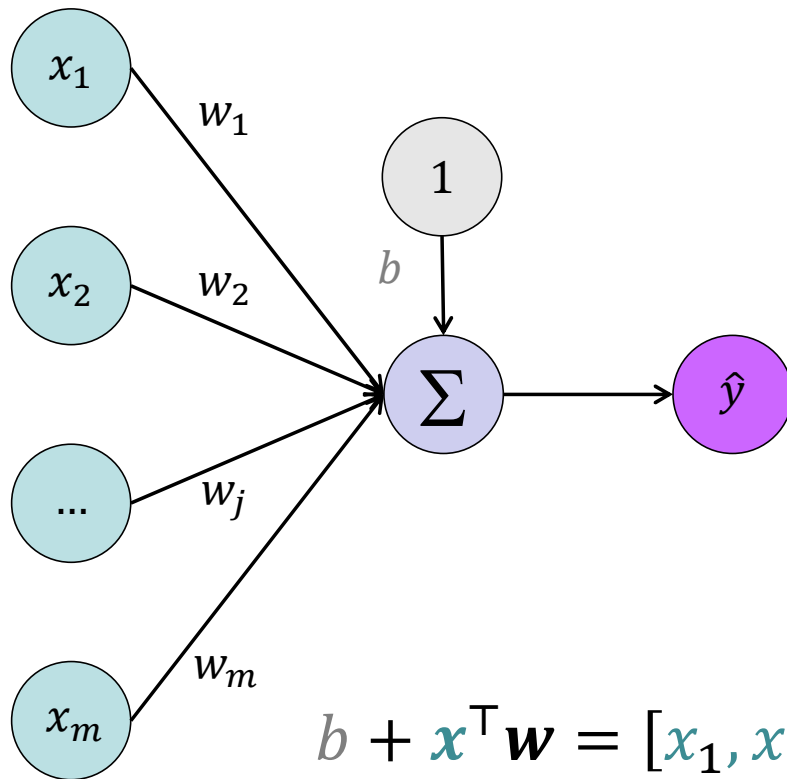
Linear Regression as Directed Graph



TRAINING DATA						
i	X_1	X_2	$\dots$	X_m	Y	$\hat{Y}$
1	x_{11}	x_{12}	$\dots$	x_{1m}	y_1	$\hat{y}_1$
2	x_{21}	x_{22}	$\dots$	x_{2m}	y_2	$\hat{y}_2$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
n	x_{n1}	x_{n2}	$\dots$	x_{nm}	y_n	$\hat{y}_n$

$$b + \mathbf{x}^\top \mathbf{w} = b + \sum_{j=1}^m w_j x_j = \hat{y}$$

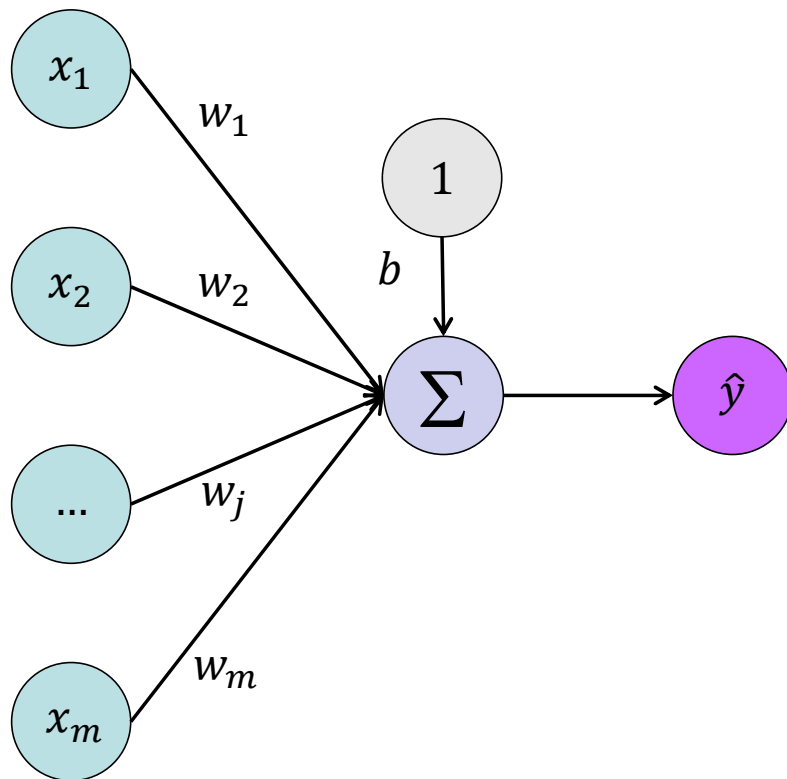
Linear Regression as Directed Graph in Matrix Notation



TRAINING DATA						
i	X_1	X_2	...	X_m	Y	$\hat{Y}$
1	x_{11}	x_{12}	...	x_{1m}	y_1	$\hat{y}_1$
2	x_{21}	x_{22}	...	x_{2m}	y_2	$\hat{y}_2$
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...	...	...	...	...	...	...
n	x_{n1}	x_{n2}	...	x_{nm}	y_n	$\hat{y}_n$

$$b + \mathbf{x}^T \mathbf{w} = [x_1, x_2, \dots, x_m] \times \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} + b = \hat{y}$$

Linear Regression as Directed Graph



Output $\hat{y}$ is calculated as the sum of the bias b and the weighted sum over the inputs:

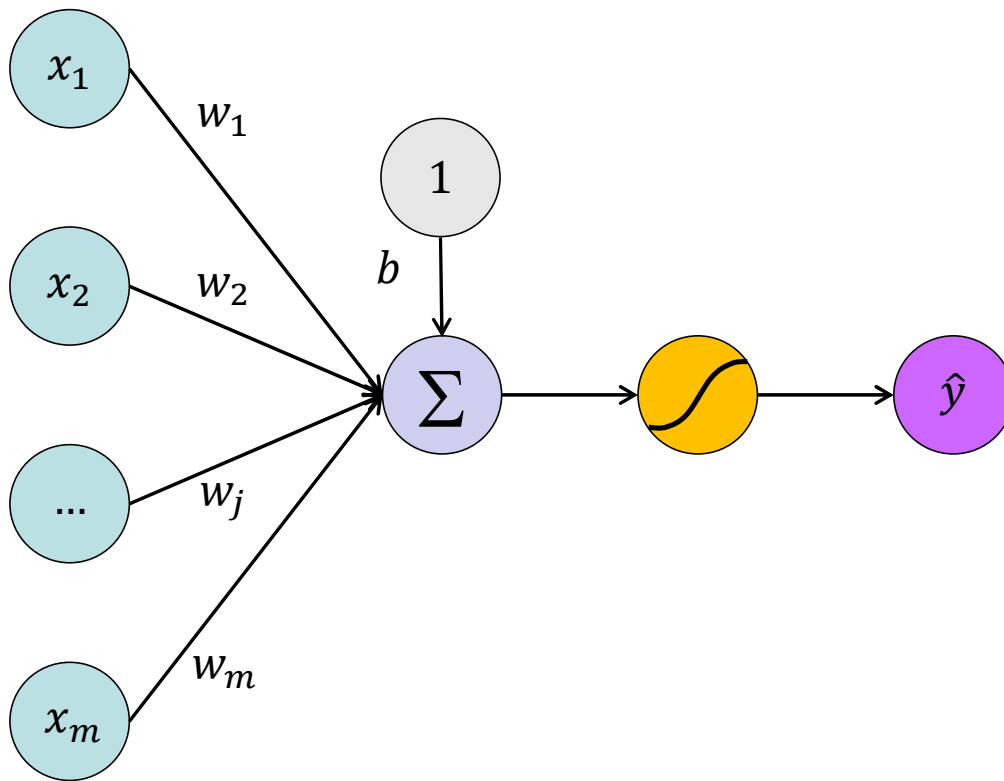
$$\hat{y} = b + \sum_{j=1}^m w_j x_j$$

The components are labeled as follows:

- $\hat{y}$: Output
- b : Bias
- $\sum_{j=1}^m w_j x_j$: Weighted sum over the inputs (Linear combination)

A Graphical Model of Generalized Linear Models (GLM)

This model is also known as Perceptron (Rosenblatt 1958)



Weighted sum over the inputs

Bias

Output

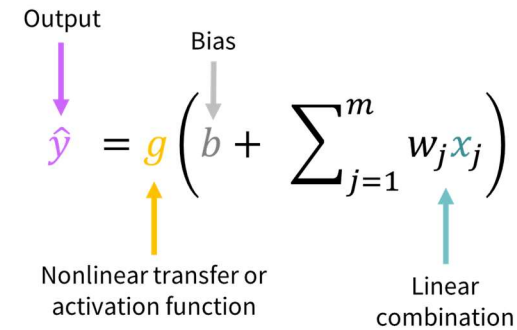
$$\hat{y} = g \left(b + \sum_{j=1}^m w_j x_j \right)$$

Nonlinear transfer or activation function

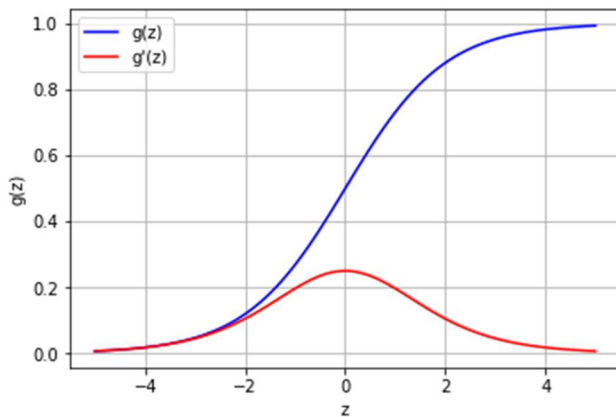
Linear combination

Common Choices for the Transfer / Activation Function

Nonlinear transformation of neuron input to neuron output



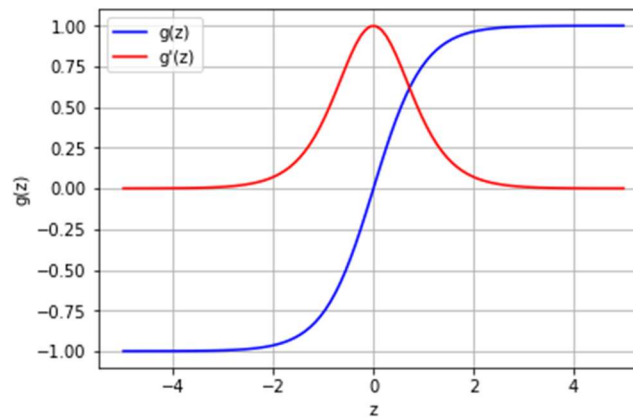
Sigmoid



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

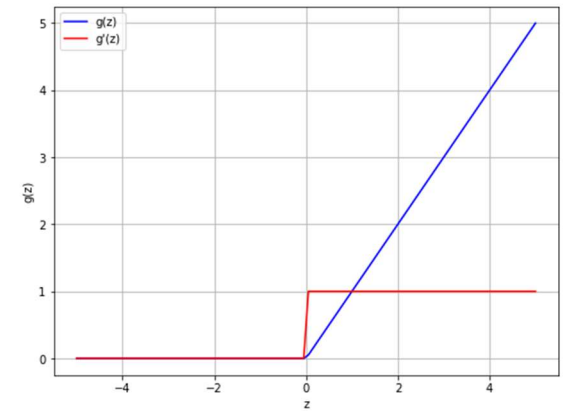
Hyperbolic tangent (tanH)



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

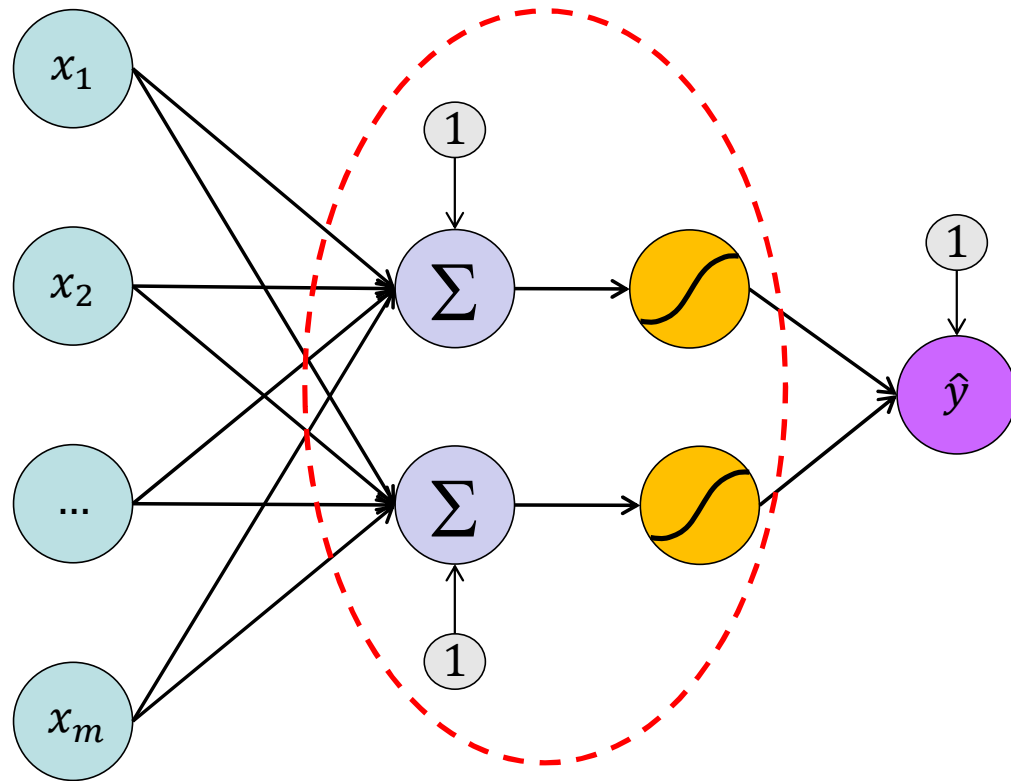
Rectified linear unit (RELU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

Neural Network (NN) with one Hidden Layer

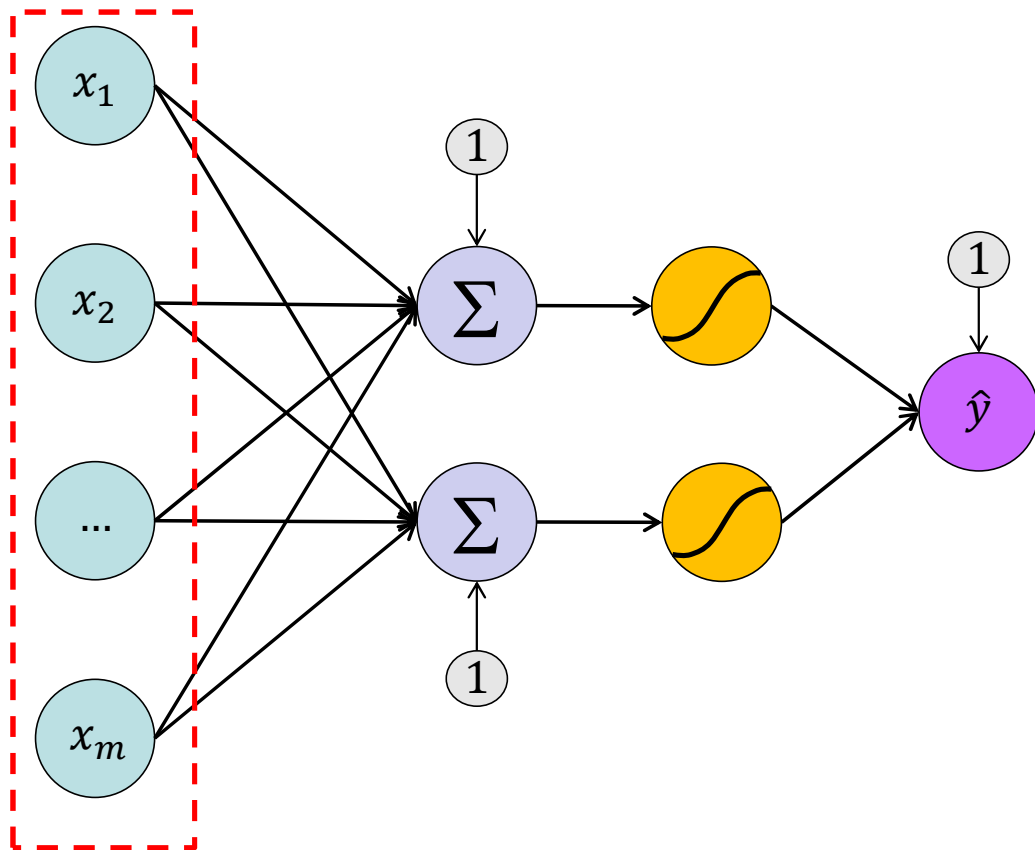


Hidden layer with $d = 2$ nodes (aka neurons)

The hidden layer (s) in a NN incorporate(s) computational units (aka hidden nodes). These hidden nodes function exactly as the perceptron introduced above. Specifically, they:

- receive **input signals** (i.e., data)
- aggregate these signals by a **weighted sum**
- put the resulting aggregate to a **transfer function**
- and pass the result on to subsequent nodes and eventually the **output node(s)**

Neural Network (NN) with one Hidden Layer – More Formally

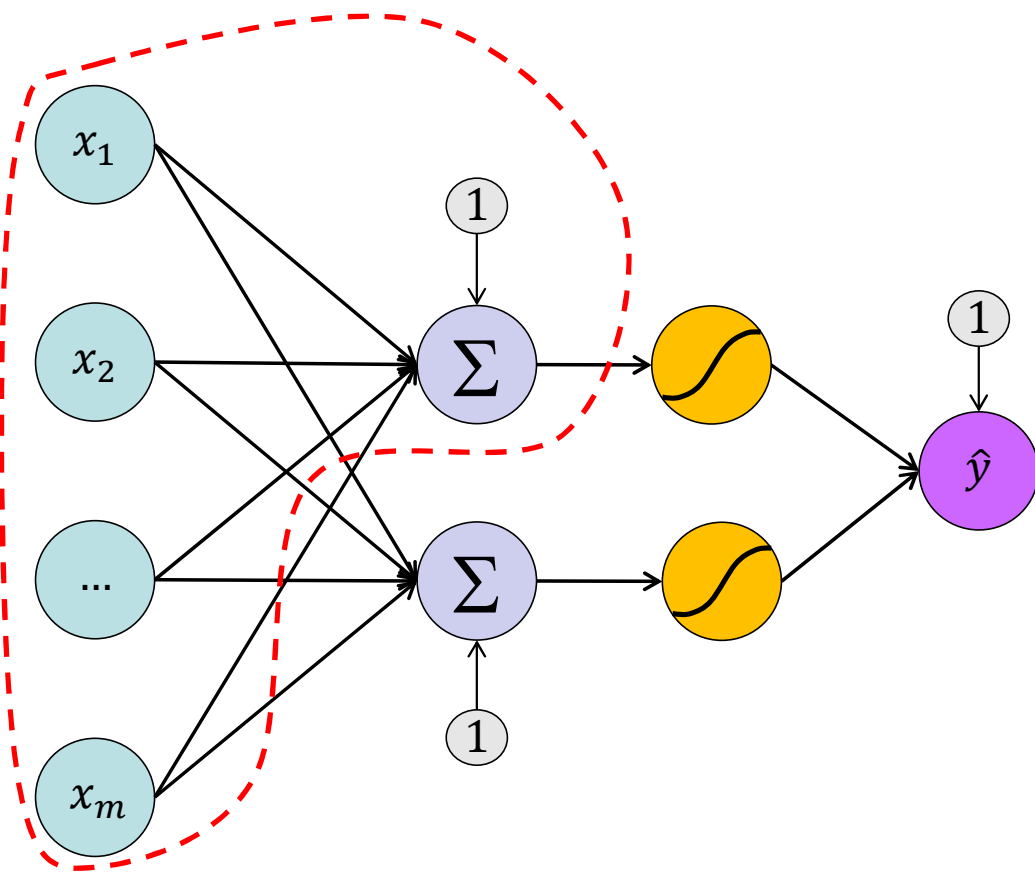


$$\mathbf{x} = (x_1, x_2, \dots, x_m)$$

$$\mathbf{x} = (2500, 36, \text{Dell XPS 15'} \dots)$$

LIST PRICE [\$]	AGE [month]	PRODUCT	...	RESALE PRICE [\$]
2,500	36	Dell XPS 15'	...	347
2,500	24	Dell XPS 15'	...	416
3,000	36	Dell XPS 17'	...	538
1,300	24	HP Envy 17'	...	121
1,900	36	HP EliteBook 850	...	172
799	12	Lenovo Yoga 11'	...	88
1,100	12	Lenovo Yoga 13'	...	266
...	...	...	...	...

Neural Network (NN) with one Hidden Layer – More Formally

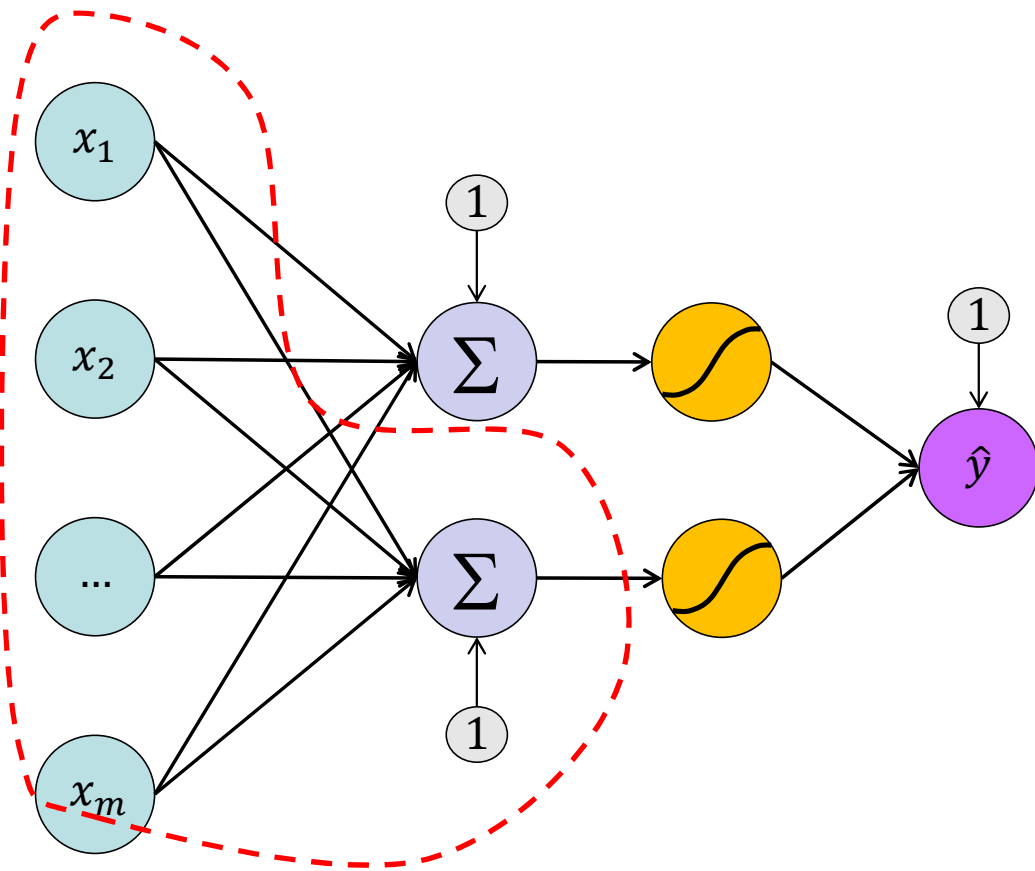


$$\mathbf{x} = (x_1, x_2, \dots, x_m)$$

$$z = b + \sum_{j=1}^m w_j x_j$$

A red dashed arrow points from the x_j term in the equation above to the x_j term in the equation below.

Neural Network (NN) with one Hidden Layer – More Formally

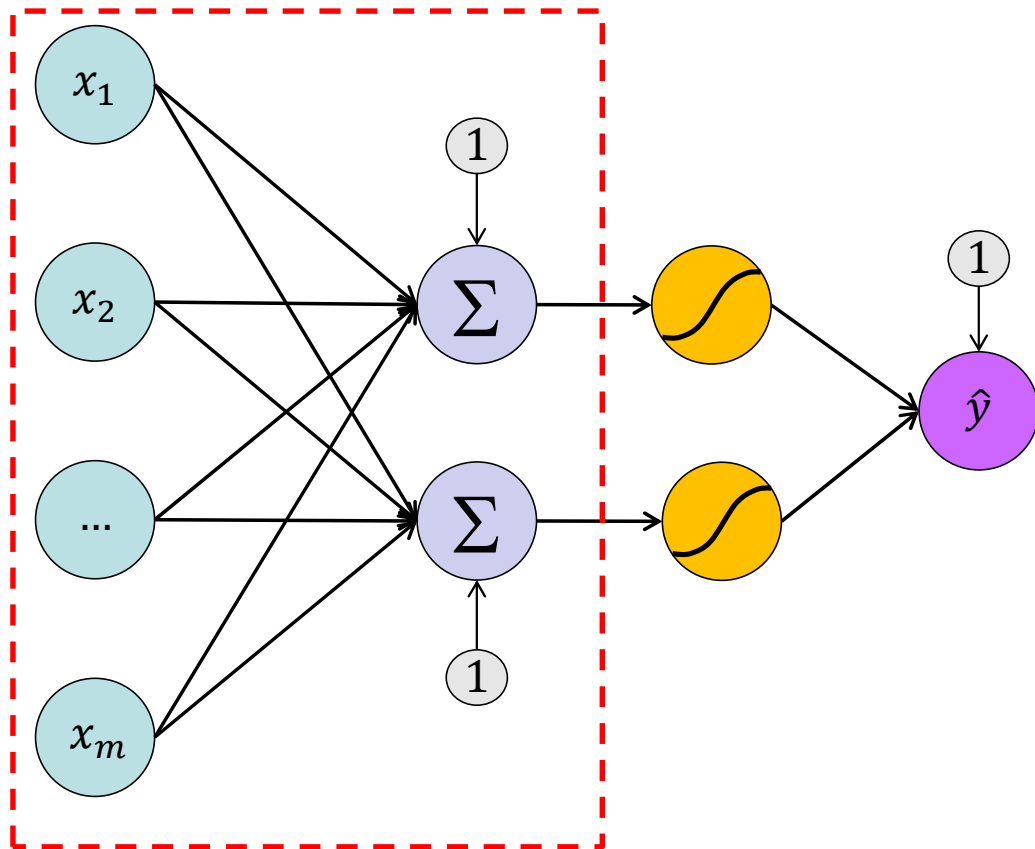


$$\mathbf{x} = (x_1, x_2, \dots, x_m)$$

$$z = b + \sum_{j=1}^m w_j x_j$$

A red dashed arrow points from the x_j term in the equation above to the x_j term in the equation below.

Neural Network (NN) with one Hidden Layer – More Formally

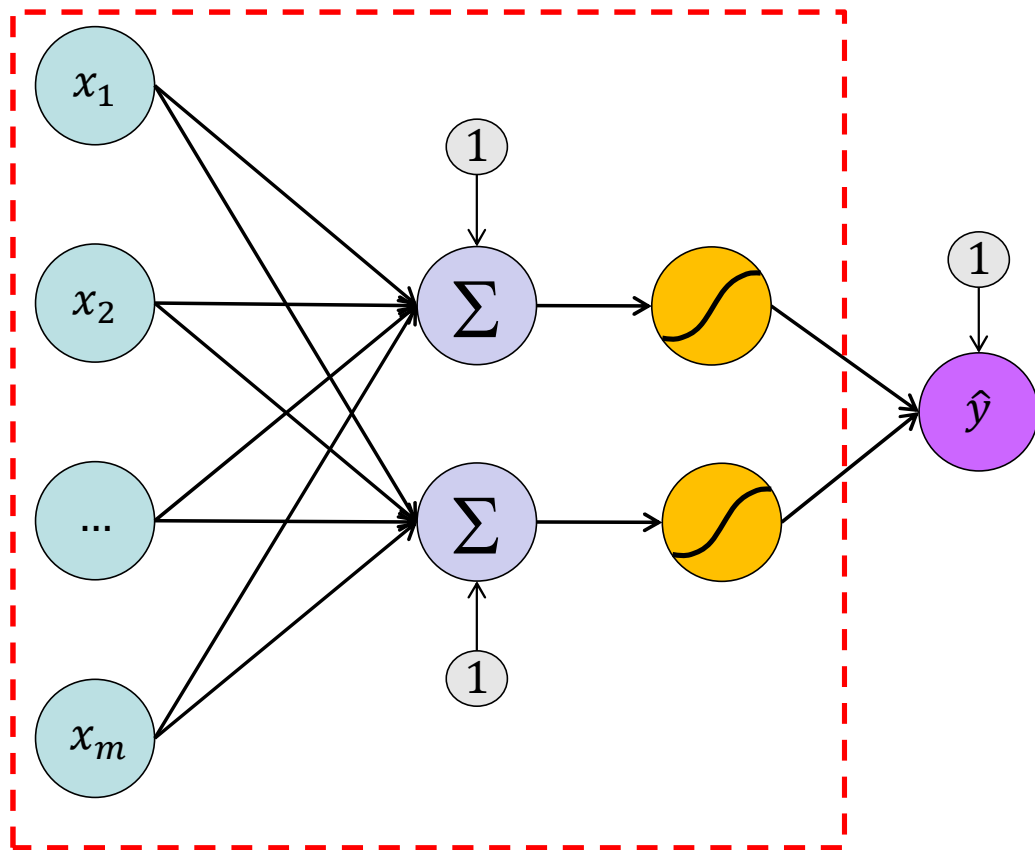


$$\mathbf{x} = (x_1, x_2, \dots, x_m)$$

$$z_k = b_k + \sum_{j=1}^m w_{kj} x_j$$

A red dashed arrow points from the x_j term in the equation above to the x_j term in the summation of the equation below.

Neural Network (NN) with one Hidden Layer – More Formally

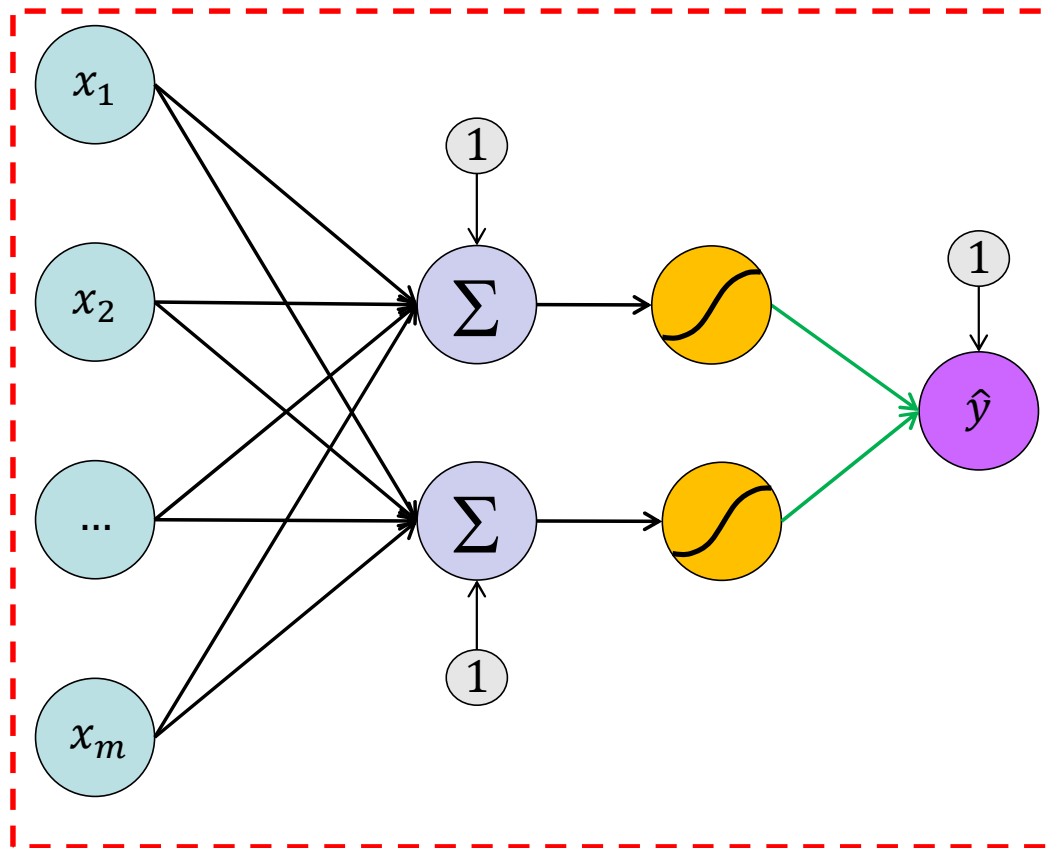


$$\mathbf{x} = (x_1, x_2, \dots, x_m)$$

$$z_k = b_k + \sum_{j=1}^m w_{kj} x_j$$

$$h_k = g(z_k)$$

Neural Network (NN) with one Hidden Layer – More Formally



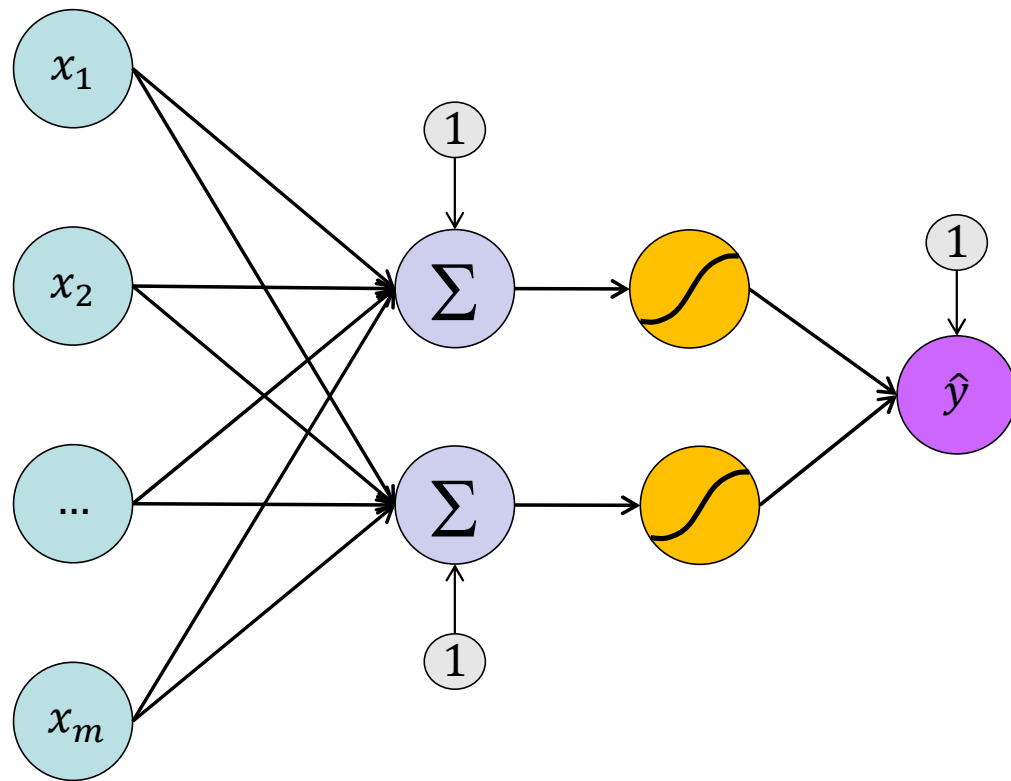
$$\mathbf{x} = (x_1, x_2, \dots, x_m)$$

$$z_k = b_k + \sum_{j=1}^m w_{kj} x_j$$

$$h_k = g(z_k)$$

$$\hat{y} = b + \sum_{k=1}^2 v_k h_k$$

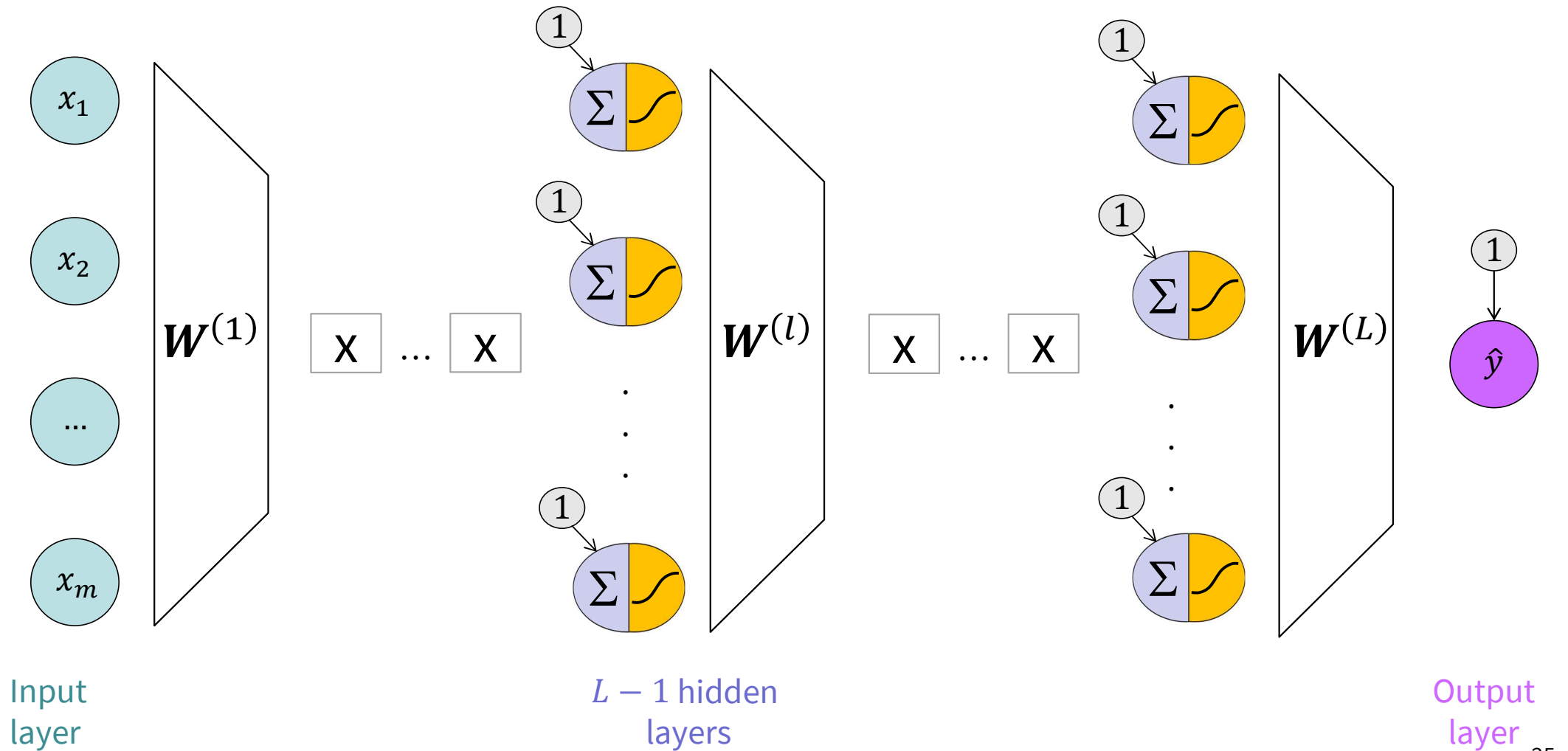
Neural Network (NN) with one Hidden Layer – More Formally



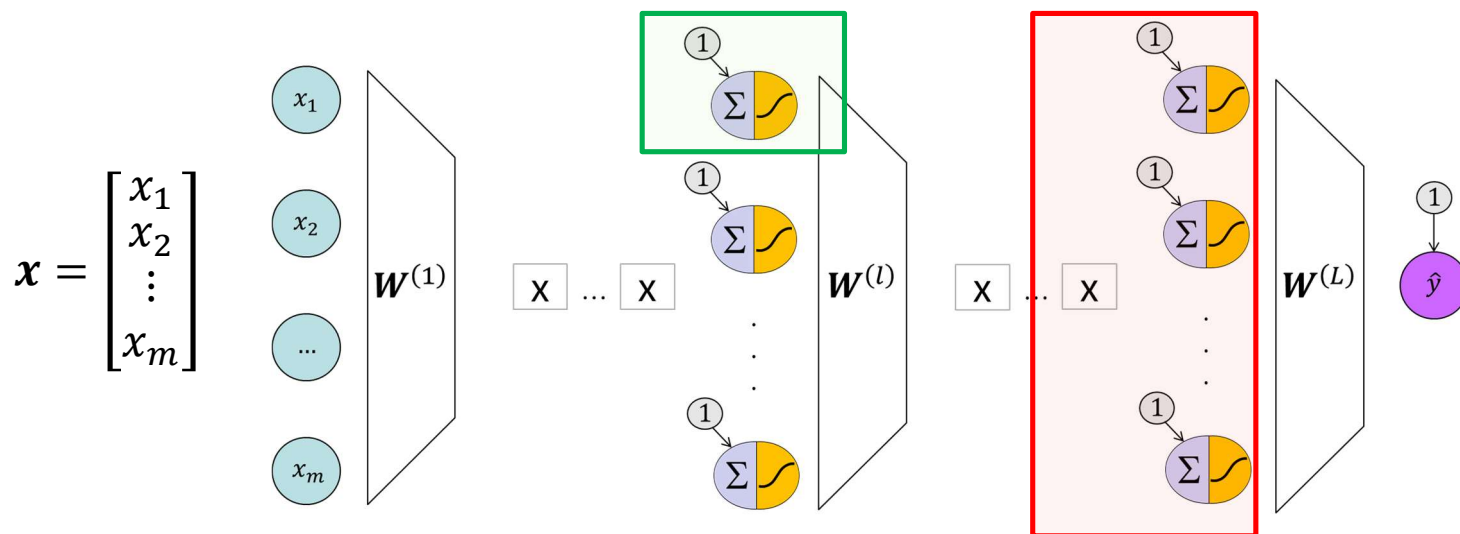
weighted sum[
nonlinearity(
weighted sum[
]
)]

$$\hat{y} = b + \sum_{k=1}^2 v_k g \left(b_k + \sum_{j=1}^m w_{kj} x_j \right)$$

Generalization to Deep Neural Networks



Formalization of the Deep Neural Network



$$z_k^{(l)} = b_k^{(l)} + \sum_{j=1}^{d_{l-1}} w_{kj}^{(l)} h_j^{(l-1)}$$

$$h_k^{(l)} = g(z_k^{(l)})$$

$$\mathbf{z}^{(l)} = \begin{bmatrix} z_1^{(l)} \\ \vdots \\ z_{d_l}^{(l)} \end{bmatrix} = \mathbf{b}^{(l)} + \mathbf{W}^{(l)} \mathbf{h}^{(l-1)}$$

$$\mathbf{h}^{(l)} = \begin{bmatrix} h_1^{(l)} \\ \vdots \\ h_{d_l}^{(l)} \end{bmatrix} = g(\mathbf{z}^{(l)}) = \begin{bmatrix} g(z_1^{(l)}) \\ \vdots \\ g(z_{d_l}^{(l)}) \end{bmatrix}$$

Notation

Data point: $\mathbf{x} = \{x_1, x_2, \dots, x_m\} \in \mathbb{R}^{1 \times m}$

Network with $L - 1$ hidden layers of d_l nodes

Node input: $z_k^{(l)}$ with $k = 1, 2, \dots, d_l$

Node bias: $b_k^{(l)}$ with $k = 1, 2, \dots, d_l$

Node output: $h_k^{(l)}$ with $k = 1, 2, \dots, d_l$

Vector of inputs to layer l : $\mathbf{z}^{(l)} \in \mathbb{R}^{d_l \times 1}$

Vector of biases of layer l : $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l \times 1}$

Vector of outputs of layer l : $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l \times 1}$

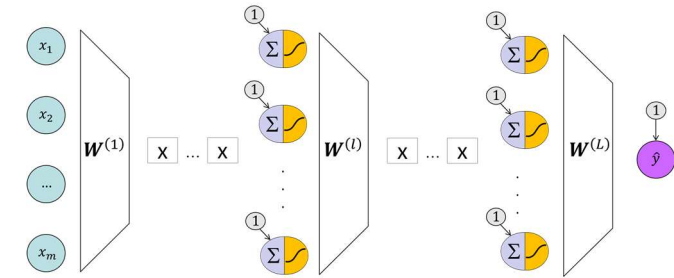
Connection weight (parameter) from node j in layer $l - 1$ to node k in layer l : w_{kj}

Weight matrix between layer $l - 1$ and l : $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$

$$\mathbf{W}^{(l)} = \begin{bmatrix} w_{11} & \cdots & w_{1d_{l-1}} \\ \vdots & \ddots & \vdots \\ w_{d_l 1} & \cdots & w_{d_l d_{l-1}} \end{bmatrix}$$

Calculation of the NN Forward Path

Illustration of computations for a **single** data point



$$\mathbf{z}^{(1)} = \mathbf{b}^{(1)} + \mathbf{W}^{(1)} \mathbf{h}^{(0)} \quad \mathbf{h}^{(1)} = g(\mathbf{z}^{(1)}) \quad \mathbf{z}^{(2)} = \mathbf{b}^{(2)} + \mathbf{W}^{(2)} \mathbf{h}^{(1)} \quad \mathbf{h}^{(2)} = g(\mathbf{z}^{(2)})$$

$$\mathbf{h}^{(0)} \equiv \mathbf{x} = \begin{bmatrix} x_1^{(1)} \\ \vdots \\ x_m^{(1)} \end{bmatrix} \quad \mathbf{z}^{(1)} = \begin{bmatrix} z_1^{(1)} \\ \vdots \\ z_{d_l}^{(1)} \end{bmatrix} \quad \mathbf{h}^{(1)} = \begin{bmatrix} g(z_1^{(1)}) \\ \vdots \\ g(z_{d_l}^{(1)}) \end{bmatrix} \quad \mathbf{z}^{(2)} = \begin{bmatrix} z_1^{(2)} \\ \vdots \\ z_{d_l}^{(2)} \end{bmatrix} \quad \mathbf{h}^{(2)} = \begin{bmatrix} g(z_1^{(2)}) \\ \vdots \\ g(z_{d_l}^{(2)}) \end{bmatrix} \quad \dots$$

$$\mathbf{h}^{(L-1)} = g(\mathbf{z}^{(L-1)}) \quad \mathbf{z}^{(L)} = \mathbf{b}^{(L)} + \mathbf{W}_{1 \times d_{l-1}}^{(L)} \mathbf{h}^{(L-1)}$$

$$\dots \quad \mathbf{z}^{(L-1)} = \begin{bmatrix} z_1^{(L-1)} \\ \vdots \\ z_{d_l}^{(L-1)} \end{bmatrix} \quad \mathbf{h}^{(L-1)} = \begin{bmatrix} g(z_1^{(L-1)}) \\ \vdots \\ g(z_{d_{L-1}}^{(L-1)}) \end{bmatrix} \quad \mathbf{z}^{(L)} = \begin{bmatrix} z_1^{(L)} \end{bmatrix} \equiv \hat{y}$$

Architecture of the Output Layer Determines NN Applications

NN are a flexible modeling framework. A few changes unlock new use cases

■ Output layer with a single node facilitates modeling of one target variable

■ Examples and use cases

□ Sales forecasting

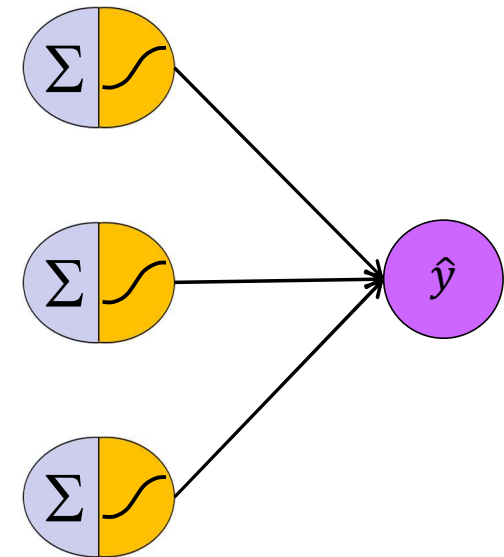
- Y : Product sales in period t
- X : Price, Last period's sales, competitor price, ...

□ Customer lifetime value (CLV)

- Y : Discounted cash flow of customer-level profit
- X : (Socio-)demographic, transactional, relational, ... data

□ Real estate pricing

- Y : Offer or sales prices of real estate
- X : Apartment size, no. rooms, floor, equipment, location data, ...



Architecture of the Output Layer Determines NN Applications

Extensions to other types of (prediction) problems

■ Neural networks provide much flexibility

- All forms of supervised learning problems are supported
- Simple regression, multi-output-regression, binary/multi-class classification, survival analysis, ...

■ Adjustment of network architecture to adapt network for given use case

- Structure of the input data (e.g., time series forecasting)
- Configuration of the output layer (e.g., multi-output regression)
- Loss function for network training (e.g., classification)

■ Hyperparameter tuning also required

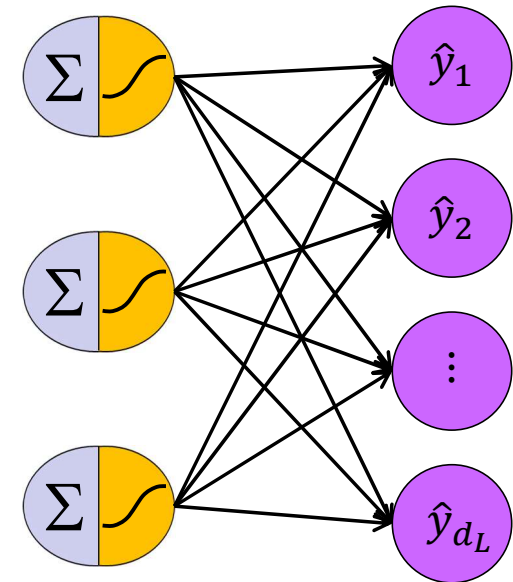
- Number and dimension of hidden layers
- Activation function
- Learning rate
- ...

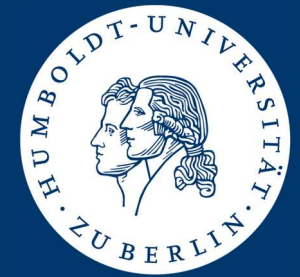
Architecture of the Output Layer Determines NN Applications

An architecture for multi-output regression

- **Output layer with $d_L > 1$ nodes, each representing the forecast for one target**
- **Train using aggregated (across targets) squared error loss**
- **Exemplary use case: financial market modeling**

- Y : Closing prices of all stocks in the S&P500 on day t
- X : Measurements of S&P500 stocks
 - Past (lagged) prices from past day $t - 1, t - 2, \dots, t - \tau$
 - Trading volume from past days
 - Technical indicators (MACD, RSI, Bollinger bands, ...)
 - Other stock indices
 - Commodity prices and other macro-economic data



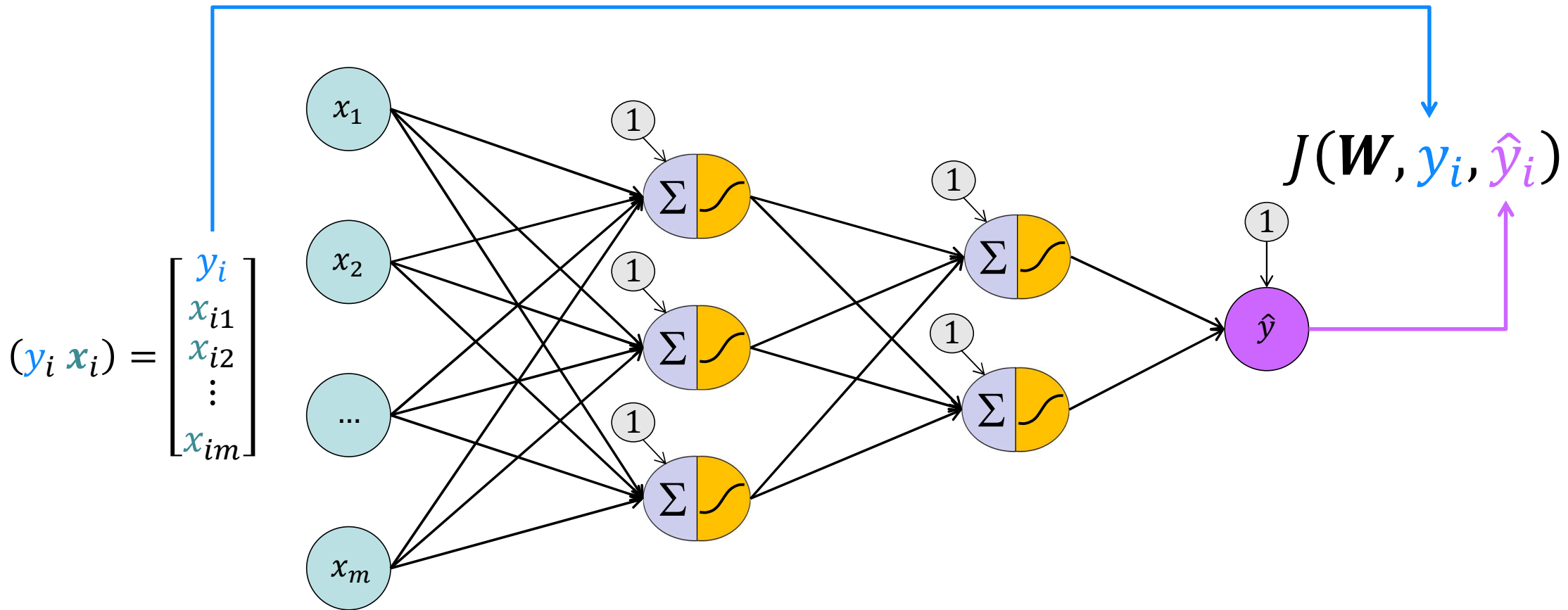


Neural Network Training

Loss function, gradient descent, and backpropagation

Neural Network Training

Loss function measures deviation between network output true target



Recap: Regression Model Estimation

Finding the optimal solution

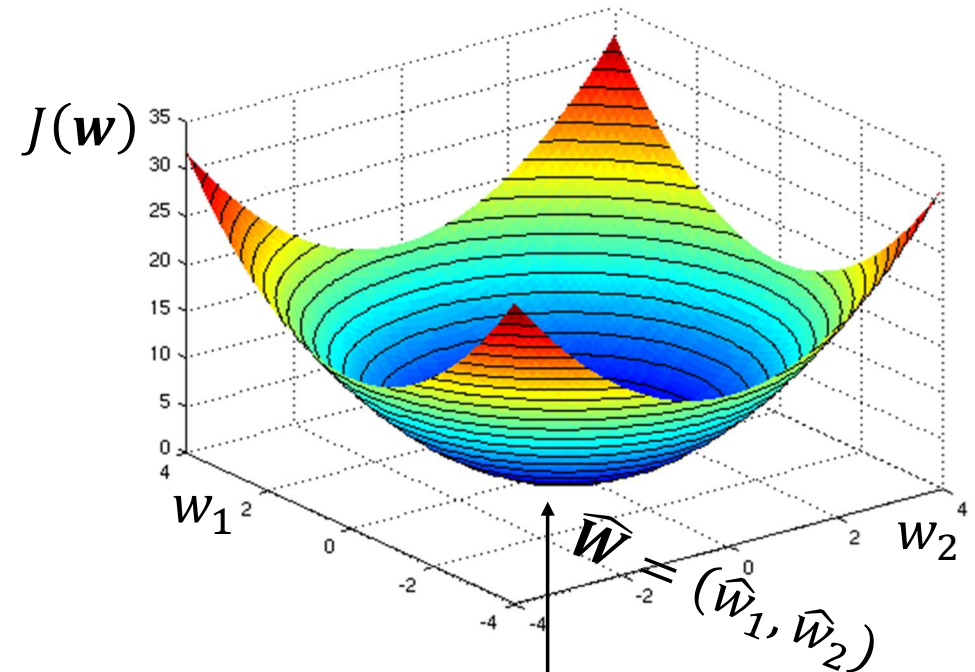
- For the simple case of linear regression with least-square loss function, we can find the optimal solution analytically

$$\square J(W) = \sum_{i=1}^n (y_i - \sum_{j=1}^m w_j x_{ij})^2$$

$$\square \hat{W} \leftarrow \operatorname{argmin}_W J(W)$$

- Calculus principle: calculate partial derivative and set to zero

$$\frac{\partial J(W)}{\partial W} = 0 \quad \left\{ \begin{array}{l} \frac{\partial J(W)}{\partial w_1} = 0 \\ \frac{\partial J(W)}{\partial w_2} = 0 \end{array} \right. \Rightarrow \hat{W} = (X^T X)^{-1} X^T y$$



Neural Network Training

Same principle as in regression but more complicated

■ Minimize loss function over free model parameters

$$\square J(\mathbf{W}) = \sum_{i=1}^n (\mathbf{y}_i - \sum_{j=1}^m w_j x_{ij})^2$$

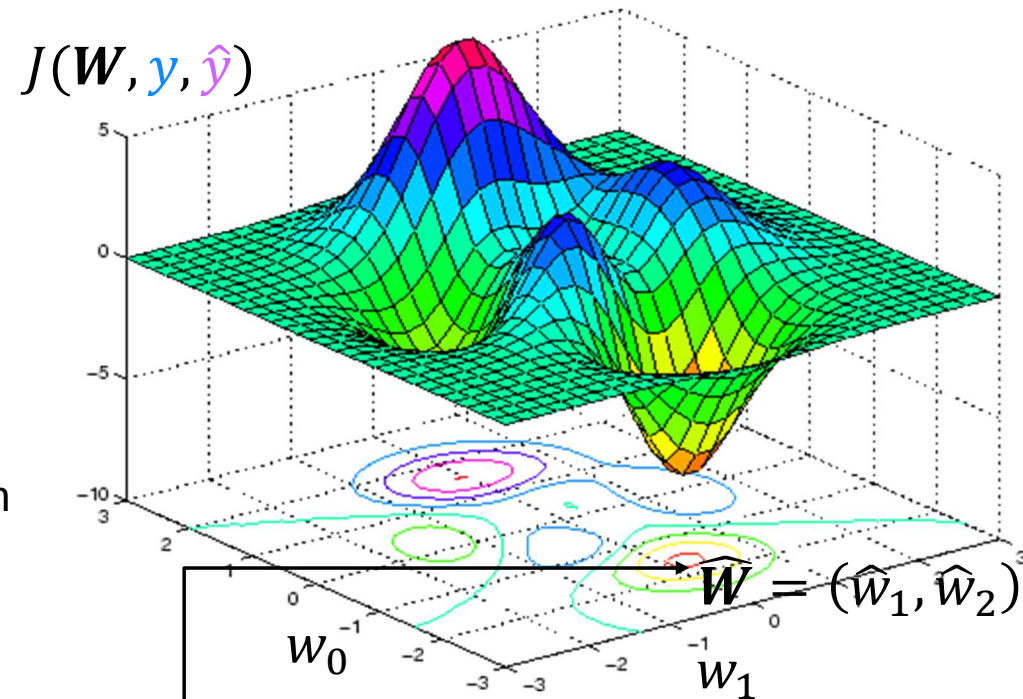
$$\square \hat{\mathbf{W}} \leftarrow \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

■ No analytical solution anymore

□ Due to added complexity in NNs

□ Multiple nodes & layers & nonlinear transfer function

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = 0 \quad \left\{ \begin{array}{l} \frac{\partial J(\mathbf{W})}{\partial w_1} = 0 \\ \frac{\partial J(\mathbf{W})}{\partial w_2} = 0 \end{array} \right. \Rightarrow \hat{\mathbf{W}} = ?$$



Neural Network Training

2D Projection of the loss surface in deep NNs for computer vision

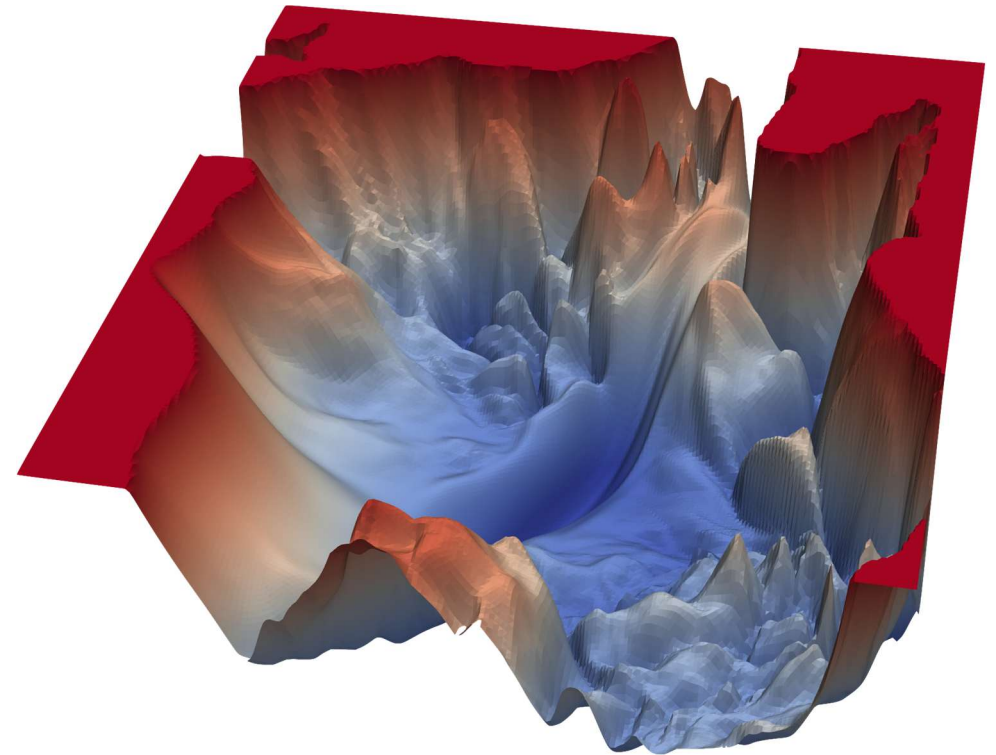
■ Minimization problem

$$\hat{W} = \underset{W}{\operatorname{argmin}} J(W, y, \hat{y})$$

■ Nonlinear, non-convex problem

■ No analytical solution

■ Training based on iterative, numerical algorithms



Source: <https://www.cs.umd.edu/~tomg/projects/landscapes/>

3D visualization tool: <https://www.telesens.co/loss-landscape-viz/viewer.html>

Paper: Hao et al., NeurIPS, 2018.

ANN Training by (Stochastic) Gradient Descent

■ Minimization problem

$$\widehat{W} = \underset{W}{\operatorname{argmin}} J(W, y, \hat{y})$$

■ Initialize weights

$$W_0 \sim N(0, \sigma^2)$$

■ Repeat

□ Compute network output
(forward path)

$$\hat{y} = f_{W_t}(x)$$

□ Compute loss gradient

$$\frac{\partial J(W_t)}{\partial W_t}$$

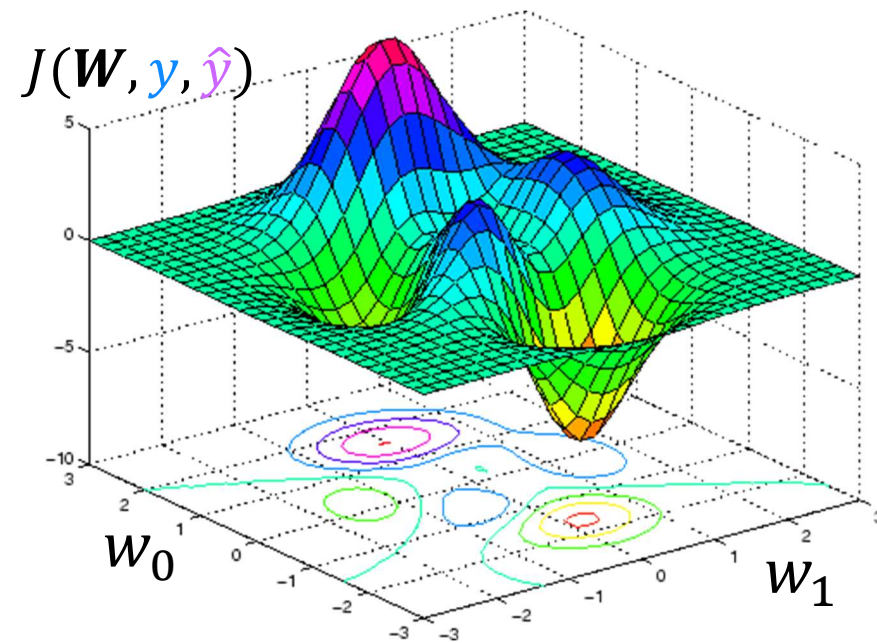
□ Update weights

$$W_{t+1} \leftarrow W_t - \eta \frac{\partial J(W_t)}{\partial W_t}$$

□ Check stopping criterion

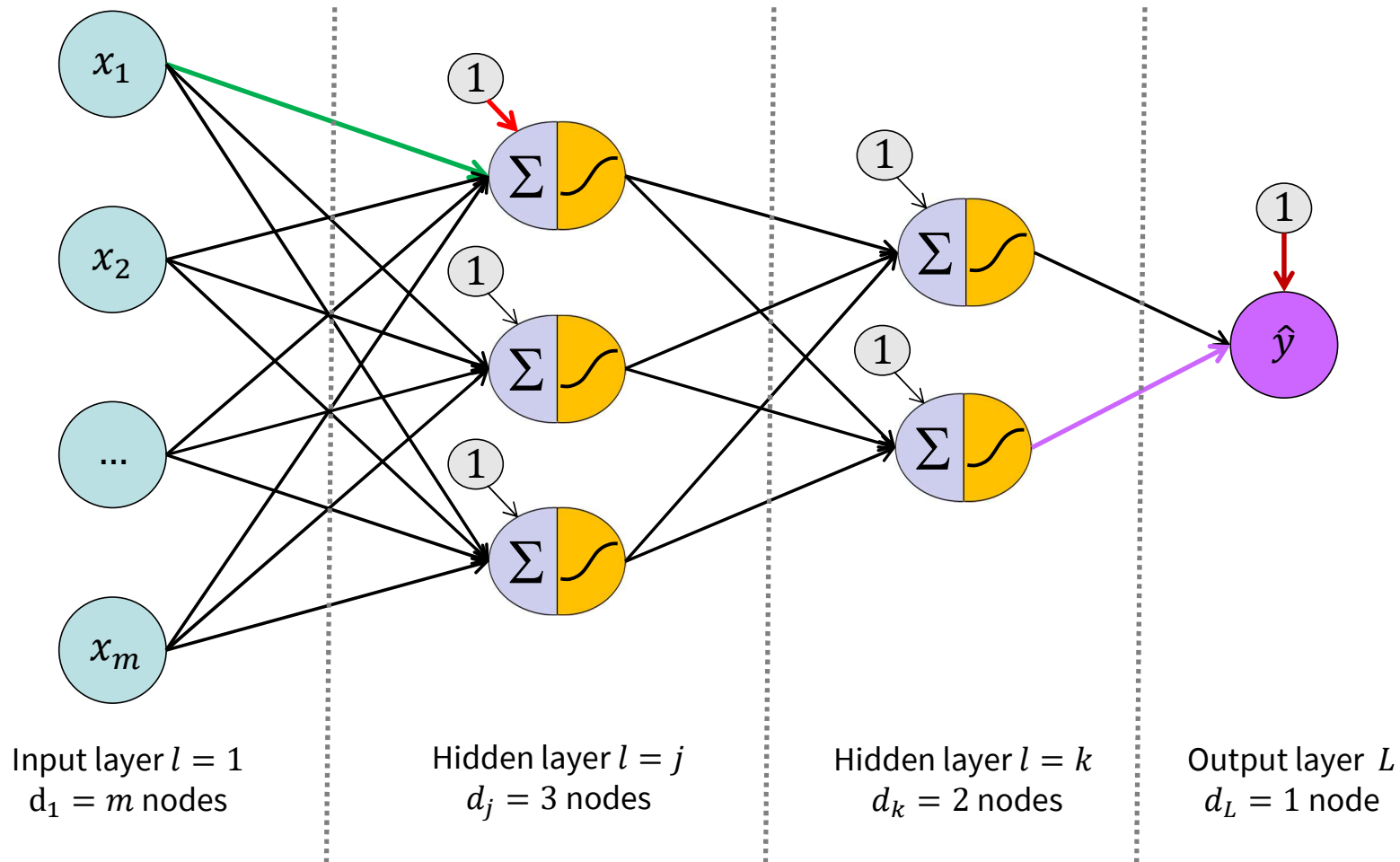
- Max number of iterations
- Change of weights

Learning rate determines how much we adjust connection weights per iteration.



The Backpropagation Algorithm

Efficient calculation of the partial derivatives of the loss function



$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial J}{\partial b_1^{(1)}} \\ \frac{\partial J}{\partial w_{11}^{(1)}} \\ \vdots \\ \frac{\partial J}{\partial b_k^{(l)}} \\ \frac{\partial J}{\partial w_{kj}^{(l)}} \\ \vdots \\ \frac{\partial J}{\partial b_{d_L}^{(L)}} \\ \frac{\partial J}{\partial w_{d_L d_{L-1}}^{(L)}} \end{bmatrix}$$

Neural Network History and Backpropagation

■ Early work starting in 1943

■ First break through: Rosenblatt's perceptron (1958)

- Neural network with one neuron in the hidden layer
- So actually logistic regression (yet with an online learning rule)

■ Second break through: Backpropagation

- Training neural networks with more than one hidden neuron
- Werbos (1974), Rumelhart et al. (1986)

■ Decline

- Lost popularity in late nineties
- Advent of support vector machines

■ Renaissance

- Several advancements since the early 2000 and after
- Deep networks for computer vision, NLP, etc.



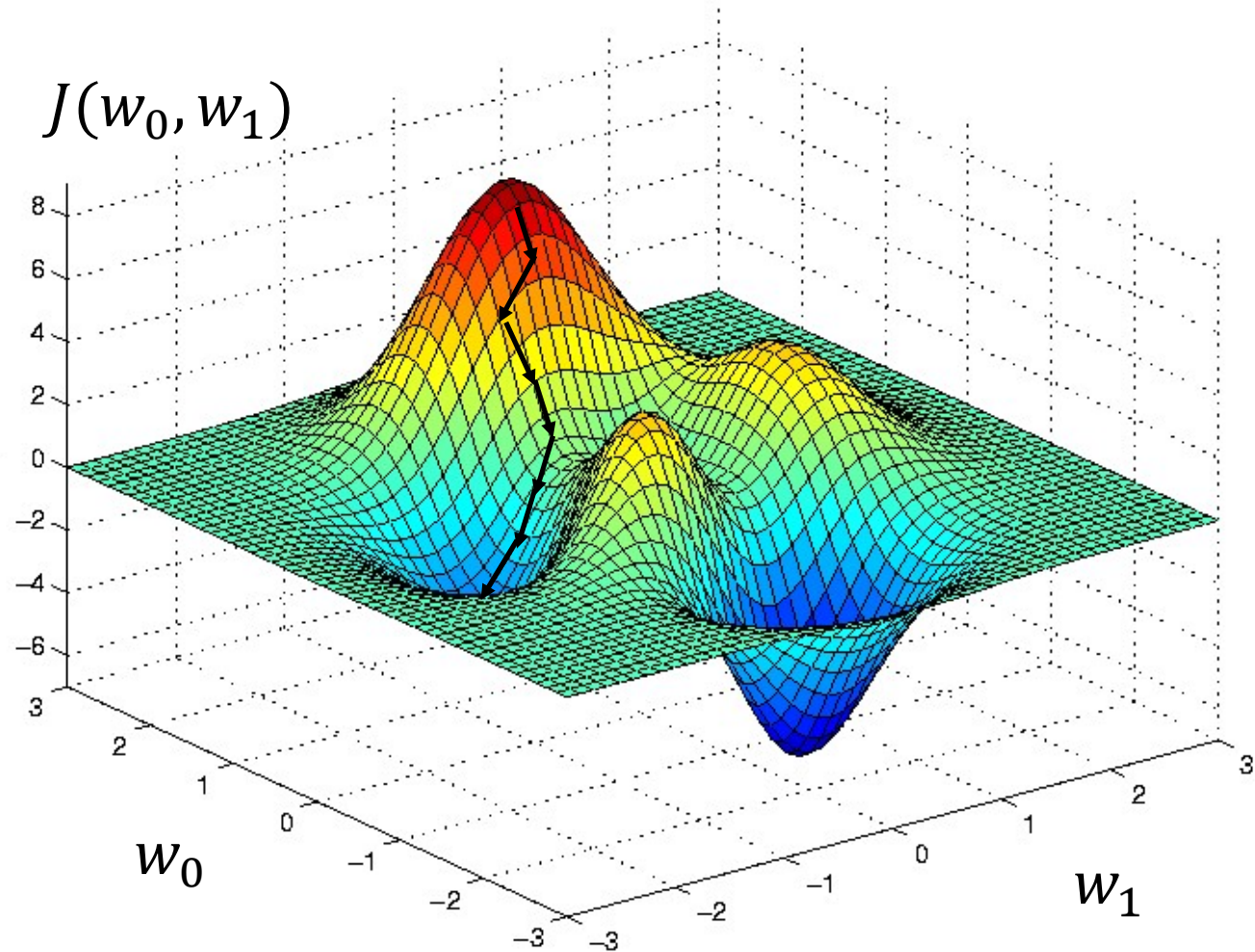
The Back-Propagation Algorithm in a Nutshell

An **iterative algorithm** that updates the weights in a neural networks in such a way that the loss is minimized. Update means that the current value of a weight is increased or decreased by a small amount.

Back-propagation is the primary algorithm for performing **gradient descent** on neural networks.

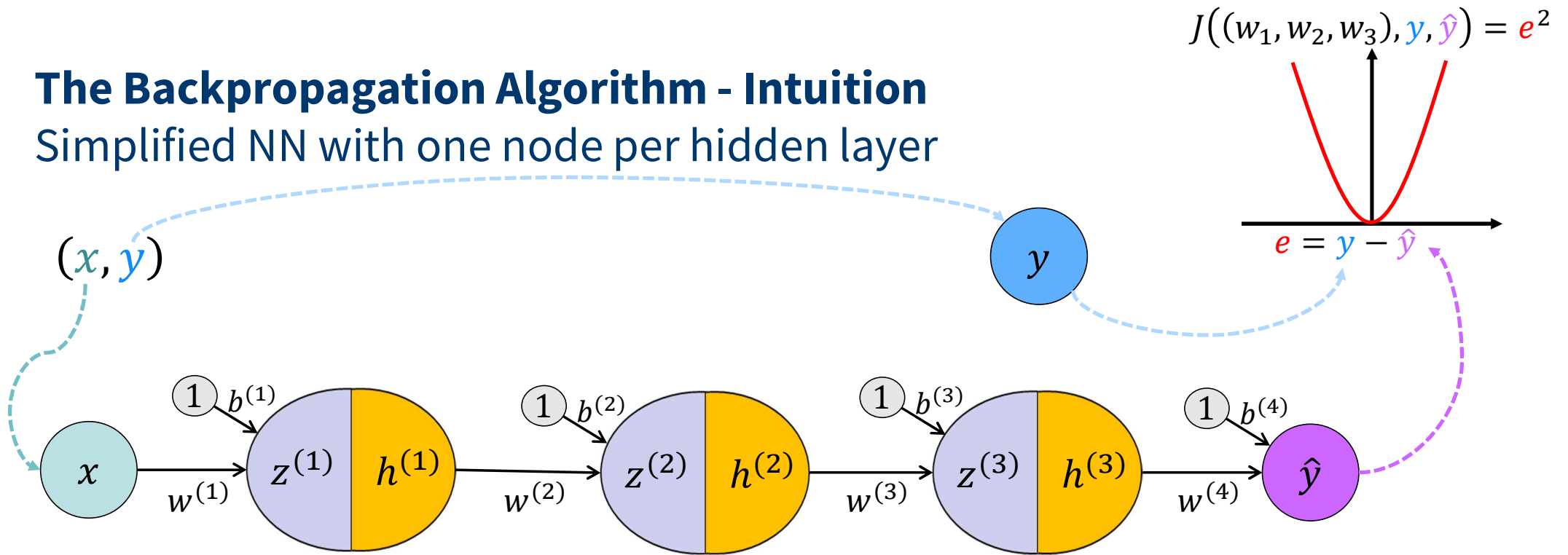
It calculates the **output of each node** in a neural network based on the current set of weights. This operation is called forward pass.

It then computes the **partial derivative** of the loss function with respect to each weight in a backward pass to identify the direction in which weights should be updated.



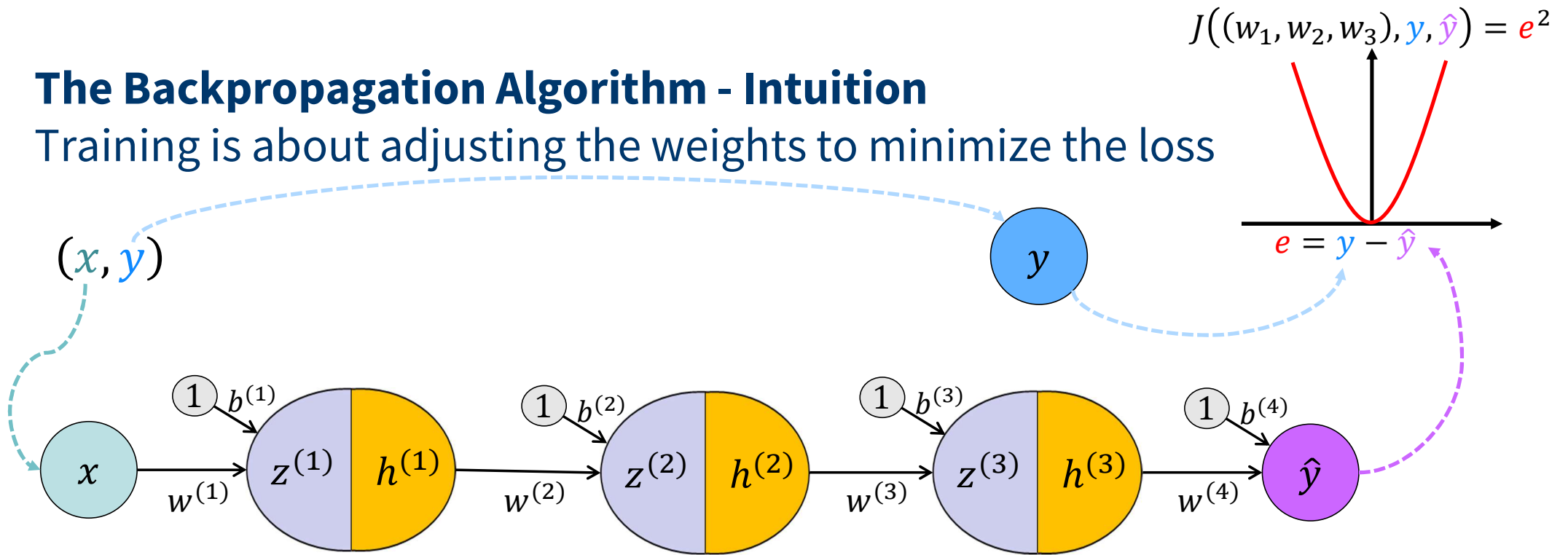
The Backpropagation Algorithm - Intuition

Simplified NN with one node per hidden layer



The Backpropagation Algorithm - Intuition

Training is about adjusting the weights to minimize the loss



■ Loss J varies with residual e

■ Residual e varies with NN output $\hat{y}$

■ NN output $\hat{y}$ varies with

□ Network weight $w^{(4)}$

□ Output of hidden unit $h^{(3)}$

■ How to adjust $w^{(4)}$ to reduce J ?

■ Examine how J changes locally with $w^{(4)}$

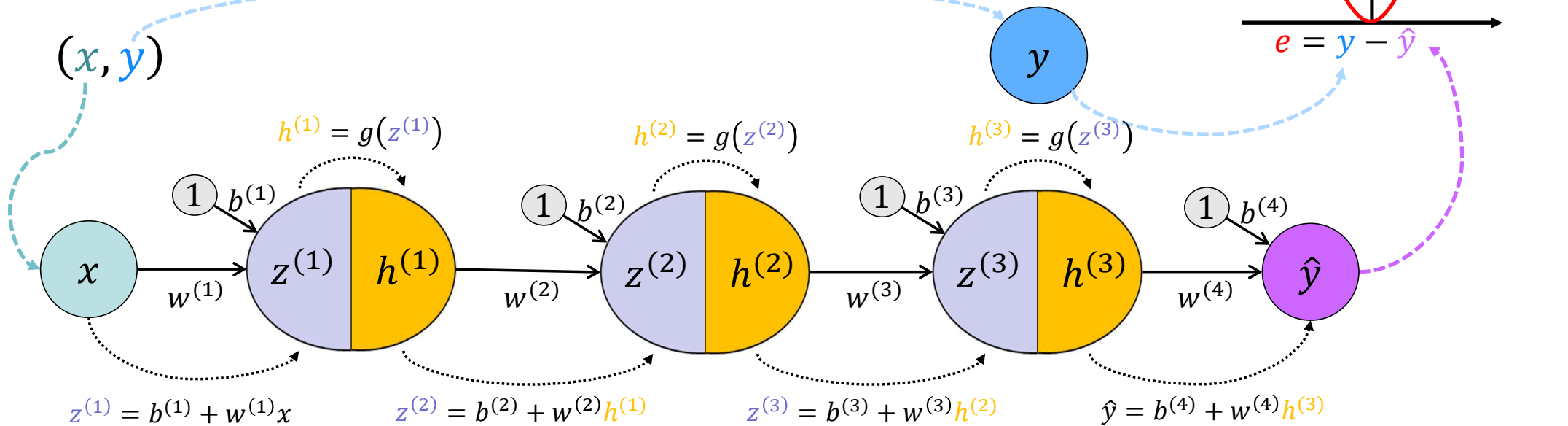
□ Formally: partial derivative of J with respect to $w^{(4)}$: $\frac{\partial J}{\partial w^{(4)}}$

□ To compute this derivative we need:

The chain rule

The Backpropagation Algorithm - Intuition

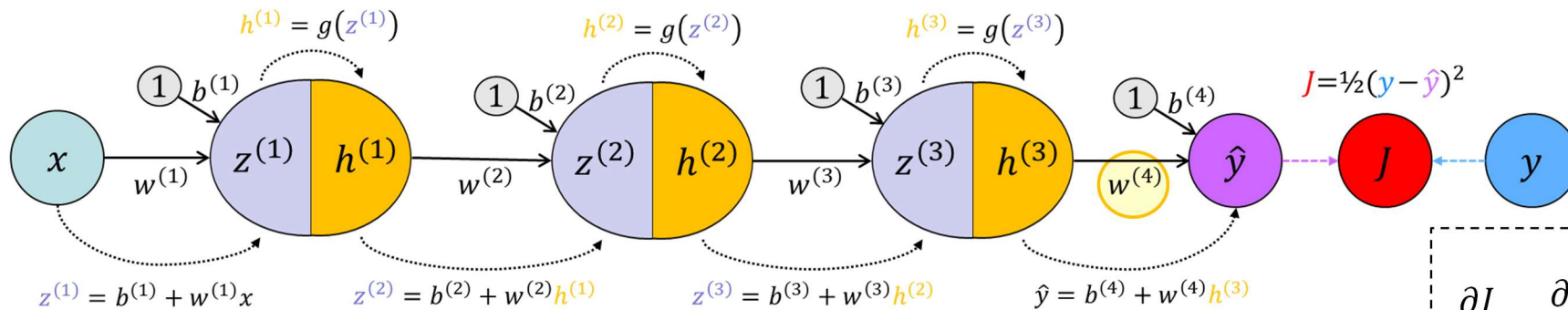
Calculation of NN output as a nested function



The Backpropagation Algorithm

Calculation of the partial derivatives in layer 4

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \begin{bmatrix} \vdots \\ \frac{\partial J}{\partial w^{(4)}} \\ \vdots \end{bmatrix}$$



$$\frac{\partial J}{\partial w^{(4)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(4)}} = (\hat{y} - y)h^{(3)} = \delta^{(4)}h^{(3)}$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{\partial \frac{1}{2}(y - \hat{y})^2}{\partial \hat{y}} = -(y - \hat{y}) = (\hat{y} - y)$$

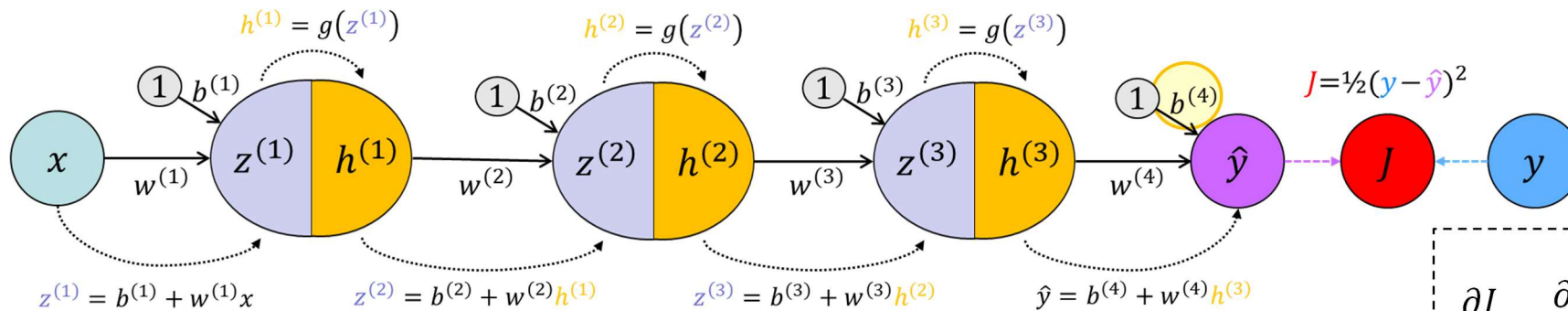
$$\frac{\partial \hat{y}}{\partial w^{(4)}} = \frac{\partial (b^{(4)} + w^{(4)}h^{(3)})}{\partial w^{(4)}} = h^{(3)}$$

$$\delta^{(4)} \equiv (\hat{y} - y)$$

The Backpropagation Algorithm

Calculation of the partial derivatives in layer 4

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \begin{bmatrix} \vdots \\ \frac{\partial J}{\partial b^{(4)}} \\ \vdots \end{bmatrix}$$



$$\frac{\partial J}{\partial w^{(4)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(4)}} = (\hat{y} - y)h^{(3)} = \delta^{(4)}h^{(3)}$$

$$\frac{\partial J}{\partial b^{(4)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial b^{(4)}} = (\hat{y} - y)1 = \delta^{(4)}$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{\partial \frac{1}{2}(y - \hat{y})^2}{\partial \hat{y}} = -(y - \hat{y}) = (\hat{y} - y)$$

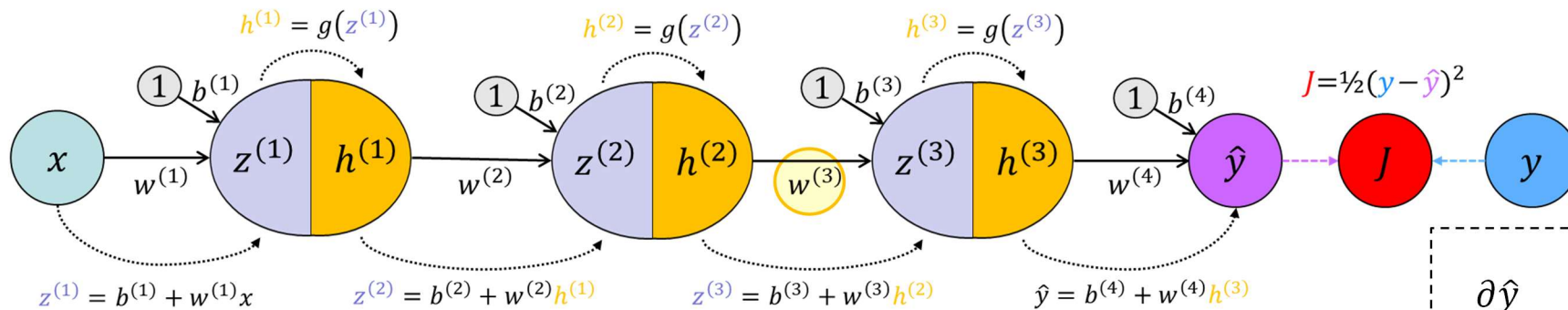
$$\frac{\partial \hat{y}}{\partial b^{(4)}} = \frac{\partial (b^{(4)} + w^{(4)}h^{(3)})}{\partial b^{(4)}} = 1$$

$$\delta^{(4)} \equiv (\hat{y} - y)$$

The Backpropagation Algorithm

Calculation of the partial derivatives in layer 3

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \begin{bmatrix} \vdots \\ \frac{\partial J}{\partial w^{(3)}} \\ \vdots \end{bmatrix}$$



$$\frac{\partial J}{\partial w^{(4)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(4)}} = (\hat{y} - y)h^{(3)} = \delta^{(4)}h^{(3)}$$

$$\frac{\partial J}{\partial w^{(3)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial z^{(3)}} \times \frac{\partial z^{(3)}}{\partial w^{(3)}} = (\hat{y} - y)w^{(4)}g'(z^{(3)})h^{(2)} = \delta^{(3)}h^{(2)}$$

$$\frac{\partial \hat{y}}{\partial h^{(3)}} = \frac{\partial (b^{(4)} + w^{(4)}h^{(3)})}{\partial h^{(3)}} = w^{(4)}$$

$$\frac{\partial h^{(3)}}{\partial z^{(3)}} = \frac{\partial (g(z^{(3)}))}{\partial z^{(3)}} = g'(z^{(3)})$$

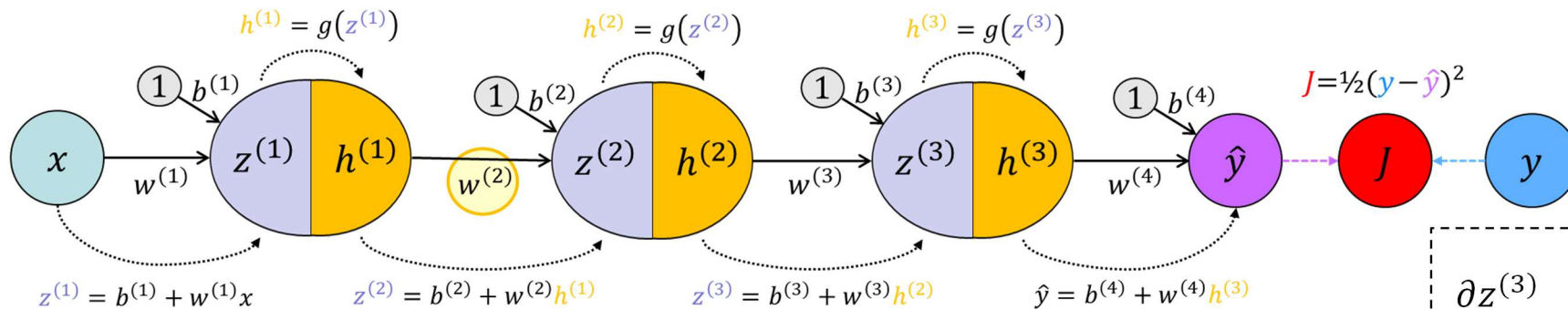
$$\frac{\partial z^{(3)}}{\partial w^{(3)}} = \frac{\partial (b^{(3)} + w^{(3)}h^{(2)})}{\partial w^{(3)}} = h^{(2)}$$

$$\delta^{(3)} \equiv \delta^{(4)}w^{(4)}g'(z^{(3)})$$

The Backpropagation Algorithm

Calculation of the partial derivatives in layer 2

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \begin{bmatrix} \vdots \\ \frac{\partial J}{\partial w^{(2)}} \\ \vdots \end{bmatrix}$$



$$\frac{\partial J}{\partial w^{(4)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(4)}} = \delta^{(4)} h^{(3)}$$

$$\frac{\partial J}{\partial w^{(3)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial z^{(3)}} \times \frac{\partial z^{(3)}}{\partial w^{(3)}} = \delta^{(3)} h^{(2)}$$

$$\frac{\partial J}{\partial w^{(2)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial z^{(3)}} \times \frac{\partial z^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(2)}}{\partial z^{(2)}} \times \frac{\partial z^{(2)}}{\partial w^{(2)}} = \dots = \delta^{(2)} h^{(1)}$$

$$\frac{\partial z^{(3)}}{\partial h^{(2)}} = \frac{\partial (b^{(3)} + w^{(3)}h^{(2)})}{\partial z^{(3)}} = w^{(3)}$$

$$\frac{\partial h^{(2)}}{\partial z^{(2)}} = \frac{\partial (g(z^{(2)}))}{\partial z^{(2)}} = g'(z^{(2)})$$

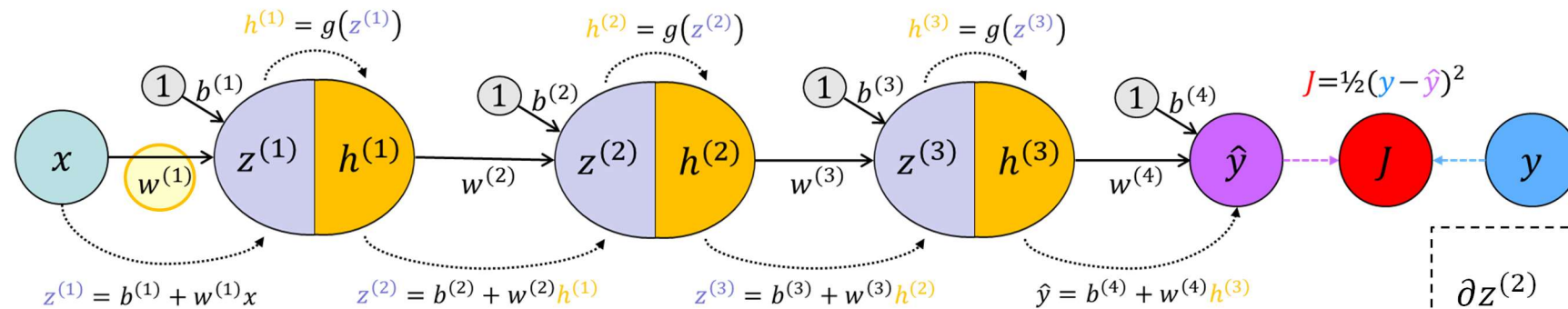
$$\frac{\partial z^{(2)}}{\partial w^{(2)}} = \frac{\partial (b^{(2)} + w^{(2)}h^{(1)})}{\partial w^{(2)}} = h^{(1)}$$

$$\delta^{(2)} \equiv \delta^{(3)} w^{(3)} g'(z^{(2)})$$

The Backpropagation Algorithm

Calculation of the partial derivatives in layer 1

$$\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \begin{bmatrix} \vdots \\ \frac{\partial J}{\partial w^{(1)}} \\ \vdots \end{bmatrix}$$



$$\frac{\partial J}{\partial w^{(4)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^{(4)}} = \delta^{(4)} h^{(3)}$$

$$\frac{\partial J}{\partial w^{(3)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial z^{(3)}} \times \frac{\partial z^{(3)}}{\partial w^{(3)}} = \delta^{(3)} h^{(2)}$$

$$\frac{\partial J}{\partial w^{(2)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial z^{(3)}} \times \frac{\partial z^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(2)}}{\partial z^{(2)}} \times \frac{\partial z^{(2)}}{\partial w^{(2)}} = \dots = \delta^{(2)} h^{(1)}$$

$$\frac{\partial J}{\partial w^{(1)}} = \frac{\partial J}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial z^{(3)}} \times \frac{\partial z^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(2)}}{\partial z^{(2)}} \times \frac{\partial z^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(1)}}{\partial z^{(1)}} \times \frac{\partial z^{(1)}}{\partial w^{(1)}} = \delta^{(1)} x$$

$$\frac{\partial z^{(2)}}{\partial h^{(1)}} = \frac{\partial (b^{(2)} + w^{(2)}h^{(1)})}{\partial z^{(2)}} = w^{(2)}$$

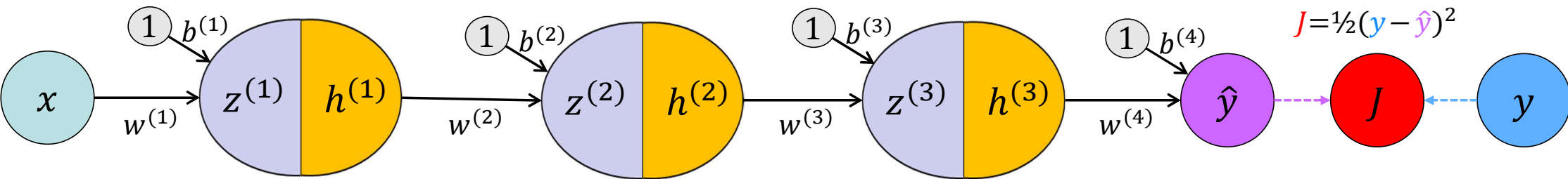
$$\frac{\partial h^{(1)}}{\partial z^{(1)}} = \frac{\partial (g(z^{(1)}))}{\partial z^{(1)}} = g'(z^{(1)})$$

$$\frac{\partial z^{(1)}}{\partial w^{(1)}} = \frac{\partial (b^{(1)} + w^{(1)}x)}{\partial w^{(1)}} = x$$

$$\delta^{(1)} \equiv \delta^{(2)} w^{(2)} g'(z^{(1)})$$

The Backpropagation Algorithm – Delta Rule

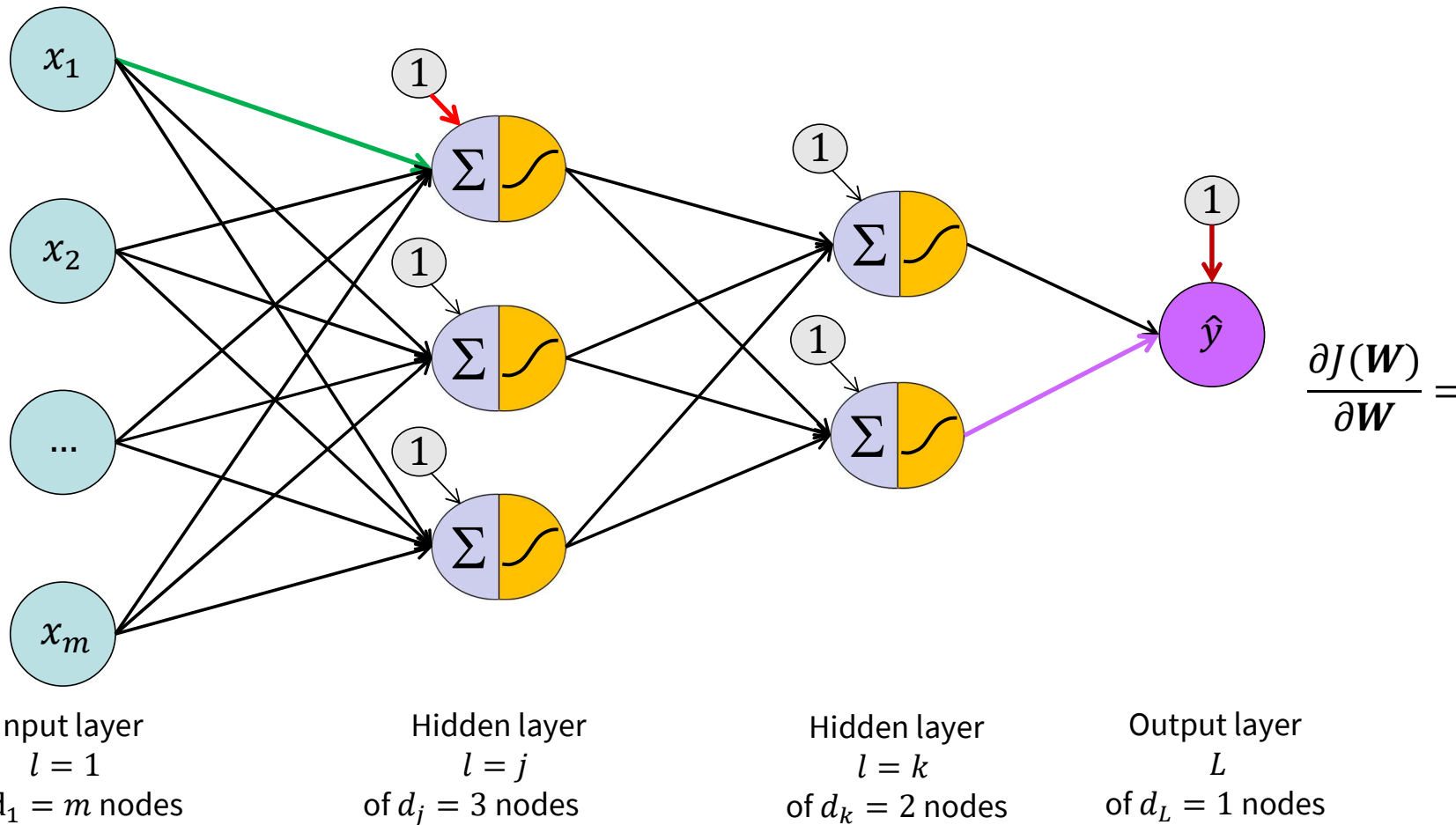
Calculation of the partial derivatives in any layer l



	Input layer $l = 1$	Hidden layers $1 < l < L$	Output layer $l = L$
Calculation of δ	$\delta^{(l)} \equiv \delta^{(l+1)} w^{(l+1)} g'(z^{(l)})$		$\delta^{(L)} = \frac{\partial J}{\partial \hat{y}}$
Partial derivative of the loss with respect to w	$\frac{\partial J}{\partial w^{(l)}} = \delta^{(l)} \times h^{(l-1)}$		
Partial derivative of the loss with respect to b	$\frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$		

The Backpropagation Algorithm – Delta Rule

Computing the gradient by backpropagating the error signal



$$\left[\begin{array}{l} \frac{\partial J}{\partial b_1^{(1)}} = \delta^{(1)} \\ \frac{\partial J}{\partial w_{11}^{(1)}} = \delta^{(1)} x \\ \vdots \\ \frac{\partial J}{\partial b_k^{(l)}} = \delta^{(l)} \\ \frac{\partial J}{\partial w_{kj}^{(l)}} = \delta^{(l)} h^{(l-1)} \\ \vdots \\ \frac{\partial J}{\partial b_{d_L}^{(L)}} = \delta^{(L)} \\ \frac{\partial J}{\partial w_{d_L d_{L-1}}^{(L)}} = \delta^{(L)} h^{(L-1)} \end{array} \right]$$

NN Training by Backpropagation & Gradient Descent

Interim Summary

- Initialize weights randomly*, e.g., $W \sim N(0, \sigma^2)$
- Loop until convergence

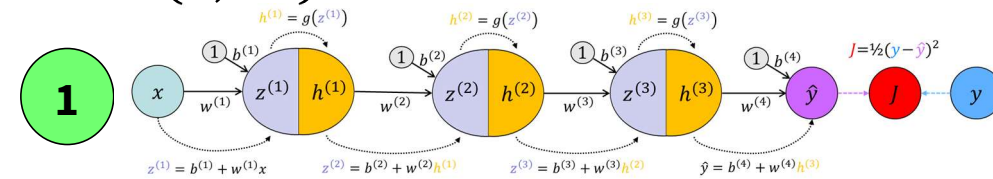
1 Compute forward path and loss

2 Compute loss and loss gradient

3 Update network weights

4 Check stopping criterion

■ Return weights



2

	Input $l = 1$	Hidden $1 < l < L$	Output $l = L$
Calculation of δ	$\delta^{(l)} \equiv \delta^{(l+1)} w^{(l+1)} g'(z^{(1)})$		$\delta^{(L)} = \frac{\partial J}{\partial \hat{y}} \times \dots$
Partial derivative of w	$\frac{\partial J}{\partial w^{(l)}} = \delta^{(l)} \times h^{(l-1)}$		
Partial derivative of b	$\frac{\partial J}{\partial b^{(l)}} = \delta^{(l)}$		

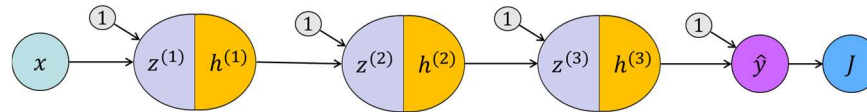
3

$$W_{t+1} \leftarrow \begin{bmatrix} w_{11}^{(1)} \\ \vdots \\ w_{kj}^{(l)} \\ \vdots \\ w_{d_L d_{L-1}}^{(L)} \end{bmatrix} - \eta \begin{bmatrix} \delta^{(1)} x \\ \vdots \\ \delta^{(l)} h^{(l-1)} \\ \vdots \\ \delta^{(L)} h^{(L-1)} \end{bmatrix}$$

*There are better ways to initialize the weights (see Glorot & Bengio 2010)

Extension: Hidden Layers Do Not Have One But Many Nodes

■ We started from here

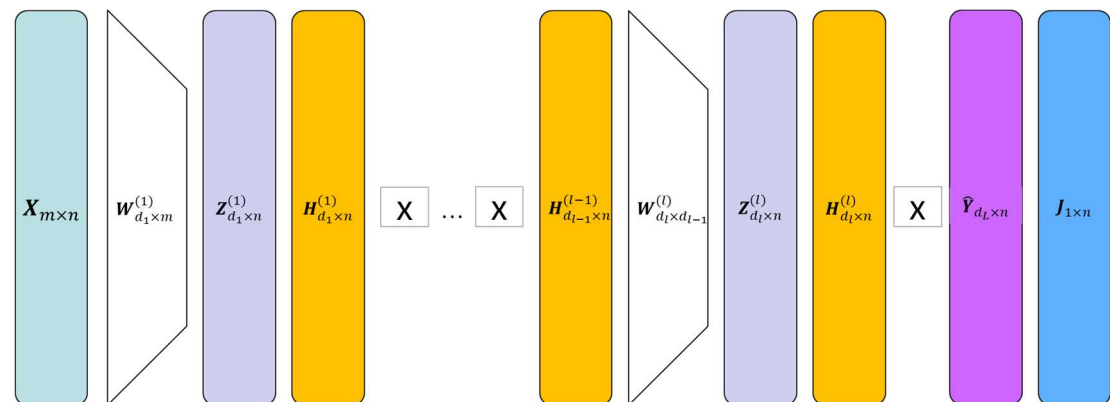


■ And found this

$$\left\{ \begin{array}{l} \frac{\partial J}{\partial w^{(l)}} = \delta^{(l)} \times h^{(l-1)} \\ \frac{\partial J}{\partial b^{(l)}} = \delta^{(l)} \end{array} \right.$$

$$\text{with } \delta^{(l)} \equiv \delta^{(l+1)} w^{(l+1)} g'(z^{(1)})$$

■ In practice, we have this



Extension: Hidden Layers Do Not Have One But Many Nodes

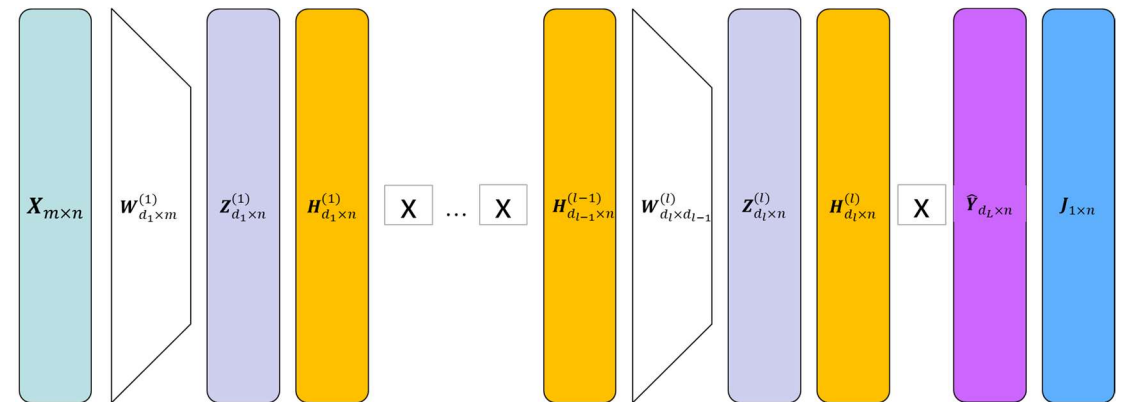
How to generalize NN training ?

■ Same principle as before

- Compute forward path and loss
- Compute gradient
- Update weights

■ Need to adjust calculation of δ

- Our $\delta^{(l)}$ from above were scalars
- Now we work with vectors and matrices



$$\delta^{(l)} \equiv \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot g'(\mathbf{Z}^{(l)})$$

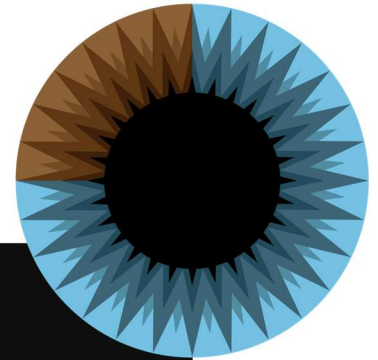
■ With the generalized $\delta^{(l)}$, we can compute the gradient just as before

■ And update the weights just as before

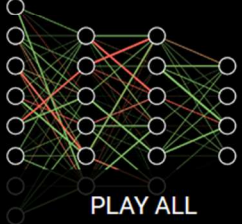
■ Calculations get somewhat tedious (see Part 3 of Salloum (2019)) if interested

Selected Resources on Backpropagation

My personal favorite: neural network series on 3Blue1Brown



Neural Networks



PLAY ALL

Neural networks

4 videos • 2,484,006 views • Last updated on Aug 1, 2018

≡+ ↗ ...

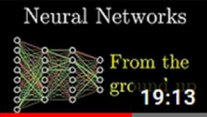


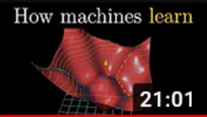
3Blue1Brown

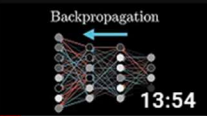
SUBSCRIBED

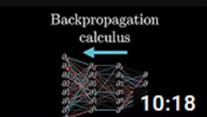


SEASON 3 ▾

- 

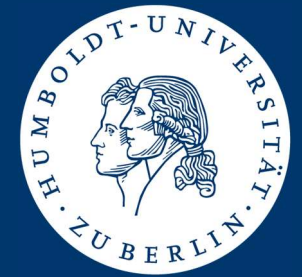
1 **Neural Networks** **But what is a neural network? | Chapter 1, Deep learning**
3Blue1Brown
19:13
- 

2 **How machines learn** **Gradient descent, how neural networks learn | Chapter 2, Deep learning**
3Blue1Brown
21:01
- 

3 **Backpropagation** **What is backpropagation really doing? | Chapter 3, Deep learning**
3Blue1Brown
13:54
- 

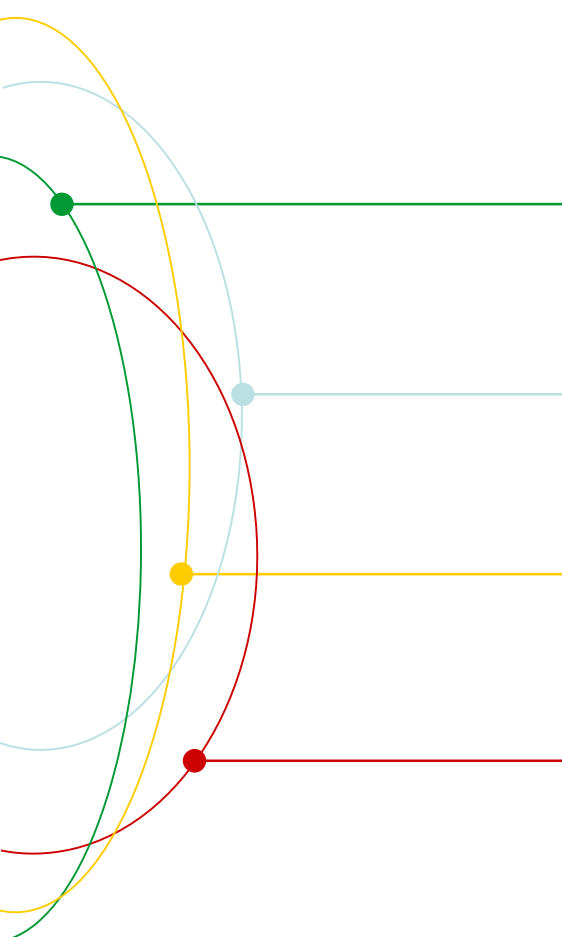
4 **Backpropagation calculus** **Backpropagation calculus | Chapter 4, Deep learning**
3Blue1Brown
10:18

Source: https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi



Summary

Summary



Learning goals

- NN foundations and building blocks
- Principles of NN training
- Interplay of NN application & architecture



Findings

- Network of inter-connected comp. units
- Training by gradient descent and back prop.
- Flexible modeling framework



Homework

- Have a look at coding examples
- Check out recommended resources
- Tutorial



What next

- More on NNs
- Tweaks and tuning

Credits & Disclaimer

- **The materials provided in the course have been inspired and draw upon numerous great sources available online**
- **The authors acknowledge the use of and wish to express their gratitude to the following sources (in alphabetical order)**
 - Machine Learning Glossary, <https://ml-cheatsheet.readthedocs.io/en/latest/index.html>
 - MIT 6.S191: Introduction to Deep Learning (<http://introtodeeplearning.com/>)
 - Medium All the Backpropagation derivatives (<https://medium.com/@pdquant/all-the-backpropagation-derivatives-d5275f727f60>)
 - Stanford CS224n: Natural Language Processing with Deep Learning (<https://web.stanford.edu/class/cs224n/>)
 - A. Zhang et al. : Dive into Deep Learning (<http://d2l.ai/index.html>)

Literature



- Rosenblatt, F. (1958) The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408.
- Salloum, Z. (2019) Back Propagation, the Easy Way. 3 Part Blog Series.
<https://towardsdatascience.com/back-propagation-the-easy-way-part-1-6a8cde653f65>

Thank you for your attention!

Stefan Lessmann

Chair of Information Systems
School of Business and Economics
Humboldt-University of Berlin, Germany

Tel. +49.30.2093.5742

Fax. +49.30.2093.5741

stefan.lessmann@hu-berlin.de

<http://bit.ly/hu-wi>

www.hu-berlin.de

